



UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO
CAMPUS JUAZEIRO
CURSO DE ENGENHARIA DA COMPUTAÇÃO

ANDRÉA CARVALHO PIRES
ANTONIO DOS SANTOS FILHO
GABRIEL MENEZES CARVALHO

DOCUMENTAÇÃO DO PROJETO DE COMPILADORES

JUAZEIRO - BA
2026

SUMÁRIO

1. LINGUAGEM-FONTE.....	4
1.1. Relação e estrutura de separadores.....	4
1.2. Sintaxe livre de contexto.....	5
1.3. Sintaxe dependente de contexto.....	6
1.3.1. Regras de escopo.....	7
1.3.2. Regras de tipos.....	7
1.4. Semântica.....	8
2. DESCRIÇÃO GERAL DA ARQUITETURA DO COMPILADOR.....	8
3. FUNDAMENTAÇÃO TEÓRICA E TÉCNICAS EMPREGADAS NA ANÁLISE SINTÁTICA.....	11
4. ANÁLISE LÉXICA.....	12
4.1. Relação de tokens.....	12
4.2. Gramática léxica.....	14
4.3. Manipulações efetuadas na gramática original e justificativas.....	15
4.4. Verificação da condição LL(1) na gramática manipulada.....	15
4.5. Técnicas utilizadas e visão geral do funcionamento.....	16
4.6. Exemplos de entradas e saídas.....	17
4.6.1. Aceitação de cadeia bem formada.....	17
4.6.2. Rejeição de cadeias mal formadas.....	19
5. ANÁLISE SINTÁTICA.....	20
5.1. Gramática sintática.....	20
5.2. Manipulações efetuadas na gramática original e justificativas.....	20
5.3. Verificação da condição LL(1) na gramática manipulada.....	22
5.4. Técnicas utilizadas e visão geral do funcionamento.....	24
5.5. Exemplos de entradas e saídas.....	24
5.5.2 Rejeição de cadeia mal formada.....	24
6. DESCRIÇÃO DAS PRINCIPAIS ESTRUTURAS DE DADOS UTILIZADAS.....	25
7. MONTAGEM E IMPRESSÃO DA ÁRVORE DE SINTAXE ABSTRATA.....	27
7.1. Estruturas de dados e algoritmos utilizados.....	27
7.2. Exemplos de programas-fonte e respectivas árvores geradas.....	28
8. ANÁLISE DE CONTEXTO.....	29
8.1. Fase de identificação: estruturas de dados e algoritmos.....	30
8.3.1 Aceitação de cadeias bem formadas.....	31
8.3.2 Rejeição de cadeias mal formadas.....	31
9. AMBIENTE DE EXECUÇÃO.....	32
9.1. Tipo de máquina usada.....	32
9.2. Avaliação de expressões.....	33

9.3. Representação de dados.....	33
9.4. Formas de alocação de memória empregadas.....	33
9.5. Passagem de parâmetros e retorno de valor de função.....	34
10. LINGUAGEM-OBJETO.....	34
10.1. Relação de instruções.....	34
10.2. Sintaxe.....	35
10.3. Semântica.....	35
11. GERAÇÃO DE CÓDIGO.....	36
11.1 Funções de código implementadas.....	36
11.2 Estruturas de dados e algoritmos utilizados.....	37
11.3 Exemplos de tradução.....	38
12. MANUAL DE COMPILAÇÃO.....	39
12.1 Requisitos de ambiente.....	40
12.2 Estrutura de pastas do projeto.....	40
12.3 Compilação no Windows.....	40
12.4 Compilação no Linux.....	40
12.5 Verificação da compilação.....	41
12.6 Script de testes (run-tests.ps1).....	41
13. MANUAL DE INSTALAÇÃO.....	42
13.1 Passo 1 — Instalar o JDK.....	42
13.2 Editor de Código:.....	43
13.3 Passo 3 — Compilar e validar.....	44
14. MANUAL DE UTILIZAÇÃO.....	45
14.1. Requisitos do sistema.....	45
14.2. Criando e localizando um arquivo-fonte.....	45
14.3. Formas de uso e opções de compilação.....	45
14.4. Onde as saídas são apresentadas.....	46
14.5. Mensagens de erro: causas e soluções.....	47
14.6. Exemplo de sessão completa de uso.....	50
15. CONCLUSÕES FINAIS.....	52
15.1. Avaliação do projeto pelo grupo.....	52
15.2. Principais dificuldades encontradas e encaminhamentos correspondentes...	53
15.3. Críticas e sugestões.....	54

1. LINGUAGEM-FONTE

A linguagem-fonte do projeto é um subconjunto da linguagem Pascal, definido a partir da gramática fornecida no enunciado do trabalho. Trata-se de uma linguagem imperativa, estruturada em blocos, com tipagem estática e dois tipos primitivos: integer (números inteiros) e boolean (valores lógicos). Um programa é composto por um cabeçalho (program nome;), uma seção opcional de declaração de variáveis, uma sequência opcional de declarações de procedimentos e funções e um comando composto principal (begin ... end), encerrado por um ponto final.

A linguagem oferece atribuição (:=), comando condicional (if/then/else), comando de repetição (while/do), comandos compostos aninhados, chamadas de procedimentos, funções com valor de retorno, expressões aritméticas sobre inteiros (+, -, *, div, mod), expressões relacionais (=, <>, <, <=, >, >=) e expressões lógicas sobre booleanos (and, or, not), além dos literais true e false. Por decisão de projeto da equipe (caso omissa da especificação), procedimentos e funções não possuem parâmetros: a comunicação entre subprogramas e programa principal ocorre por meio das variáveis do programa, e o valor de retorno de uma função é definido por atribuição ao próprio nome da função dentro do seu corpo, como no Pascal tradicional.

O exemplo a seguir ilustra os principais recursos da linguagem (arquivo exemplos/procedimento-funcao.txt):

```
program Subprogramas;
var
  x: integer;
  ok: boolean;

procedure incrementar;
begin
  x := x + 1
end;

function positivo: boolean;
begin
  positivo := x > 0
end;

begin
  x := 1;
  incrementar;
  ok := positivo
end.
```

1.1. Relação e estrutura de separadores

Na linguagem, separadores são trechos do texto-fonte que não geram tokens: servem apenas para delimitar visualmente os elementos do programa e são descartados pelo analisador léxico antes do reconhecimento de cada token. São

separadores: o espaço em branco, o caractere de tabulação, as quebras de linha (LF e CRLF) e os comentários.

Três formas de comentário são aceitas, todas descartadas integralmente:

Tabela 1 – Formas de comentário aceitas pela linguagem

Forma	Delimitadores	Alcance
Comentário de chaves	{ ... }	Do caractere { até o próximo } (pode ocupar várias linhas)
Comentário de parêntese-asterisco	(* ... *)	Do par (* até o próximo par *) (pode ocupar várias linhas)
Comentário de linha	// ...	Do par // até o fim da linha corrente

É importante destacar que os símbolos de pontuação ; : , . () não são separadores: são tokens da linguagem, com papel sintático próprio (fim de comando, separador de declaração de tipo, separador de lista de identificadores, fim de programa e agrupamento de expressões, respectivamente). Comentários abertos e não fechados ({ sem }, ou (* sem *)) constituem erro léxico, reportado com linha e coluna.

1.2. Sintaxe livre de contexto

A sintaxe livre de contexto da linguagem, na forma em que foi efetivamente implementada (já transformada para atender à condição LL(1), conforme discutido nas Seções 4 e 5), é dada pela gramática abaixo, em notação EBNF. Símbolos entre aspas são terminais; ? indica opcionalidade; * indica zero ou mais repetições; + indica uma ou mais repetições.

```
Programa ::= 'program' IDENT ';' Bloco '.'
Bloco    ::= SecaoVar? Subprograma* CompostoCmd
SecaoVar ::= 'var' DeclVar+
DeclVar  ::= ListIdent ':' Tipo ';'
```

```

ListIdent ::= IDENT (';' IDENT)*
Tipo      ::= 'integer' | 'boolean'

Subprograma ::= Procedimento | Funcao
Procedimento ::= 'procedure' IDENT ';' Bloco ';'
Funcao      ::= 'function' IDENT ':' Tipo ';' Bloco ';'

CompostoCmd ::= 'begin' (Cmd ';' Cmd)*? 'end'
Cmd          ::= AtribCmd | ChamadaProc | SeCmd | EnquantoCmd
              | CompostoCmd | vazio
AtribCmd     ::= IDENT ':=' Expr
ChamadaProc  ::= IDENT
SeCmd        ::= 'if' Expr 'then' Cmd ('else' Cmd)?
EnquantoCmd  ::= 'while' Expr 'do' Cmd

Expr         ::= ExpSimples (OpRel ExpSimples)?
OpRel        ::= '=' | '<>' | '<' | '<=' | '>' | '>='
ExpSimples   ::= Termo (('+' | '-' | 'or') Termo)*
Termo        ::= Fator (('*' | 'div' | 'mod' | 'and') Fator)*
Fator        ::= IDENT | LITERAL_INTEIRO | LITERAL_TEXTO
              | 'true' | 'false' | '(' Expr ')'
              | 'not' Fator | '-' Fator

```

A estratificação das expressões em três níveis (Expr, ExpSimples e Termo) codifica diretamente na gramática a precedência dos operadores: operadores relacionais têm a menor precedência, seguidos dos aditivos (+, -, or) e, por fim, dos multiplicativos (*, div, mod, and), que têm a maior precedência entre os binários. Os operadores unários not e - (negação aritmética) aplicam-se a fatores e, portanto, têm precedência ainda maior. Parênteses permitem sobrepor a precedência padrão. Os operadores binários de mesmo nível associam à esquerda, consequência direta da forma iterativa das produções de ExpSimples e Termo.

1.3. Sintaxe dependente de contexto

As dependências de contexto são as regras que não podem ser expressas por uma gramática livre de contexto e que, por isso, são verificadas em fase própria (a análise de contexto, Seção 8). Na linguagem do projeto elas se dividem em regras de escopo e regras de tipos.

1.3.1. Regras de escopo

Todo identificador (variável, procedimento ou função) deve ser declarado antes de ser usado. Um mesmo identificador não pode ser declarado duas vezes. Por decisão da equipe (caso omissso), o escopo é único e global: as variáveis declaradas na seção var do programa são visíveis em todo o programa, inclusive dentro dos corpos de procedimentos e funções; os nomes de subprogramas também habitam esse espaço único de nomes. Um procedimento só pode ser usado como comando (chamada); uma função só pode ser usada como fator em expressões; o nome de uma função só pode receber atribuição dentro do corpo da própria função (é assim que o valor de retorno é definido); um procedimento nunca pode receber atribuição nem ser usado em expressão.

1.3.2. Regras de tipos

A linguagem é fortemente tipada e não realiza conversões implícitas. As regras aplicadas a cada operador e comando são resumidas na tabela a seguir.

Tabela 2 – Regras de tipos da linguagem

Construção	Exigência sobre operandos	Tipo resultante
+ - * div mod	inteiro × inteiro	inteiro
- unário	inteiro	inteiro
and or	booleano × booleano	booleano
not	booleano	booleano
< <= > >=	inteiro × inteiro	booleano
= <>	operandos do mesmo tipo	booleano
v := E	tipo de E igual ao tipo declarado de v	—
if E then ... / while E do ...	E deve ser booleana	—
f := E (dentro de f)	tipo de E igual ao tipo de retorno de f	—

Um caso omissso definido pela equipe merece registro: o analisador léxico reconhece literais de texto entre aspas simples ('exemplo') e a gramática sintática os aceita como fator; porém, como a linguagem só possui os tipos integer e boolean, o analisador de contexto rejeita qualquer uso de literal de texto, com a mensagem “literal de texto nao e suportado: a linguagem aceita apenas 'integer' e 'boolean'”.

Assim, o token existe na estrutura léxica, mas nenhum programa válido pode utilizá-lo.

1.4. Semântica

A semântica da linguagem é a usual das linguagens imperativas. Um programa executa o seu comando composto principal do início ao fim, sequencialmente. A atribuição $v := E$ avalia a expressão E e armazena o resultado na variável v . O comando `if E then C1 else C2` avalia a condição E : se verdadeira, executa $C1$; se falsa, executa $C2$ (ou nada, quando a cláusula `else` está ausente). O comando `while E do C` avalia E antes de cada iteração e executa C enquanto E for verdadeira; se E for falsa logo na primeira avaliação, C não é executado nenhuma vez. A cláusula `else` sempre se associa ao `if` mais próximo (regra do *dangling else*).

Expressões são avaliadas com os operandos na ordem da esquerda para a direita, respeitando a precedência e a associatividade descritas na Seção 1.2. Os operadores lógicos `and` e `or` avaliam sempre os dois operandos (não há curto-circuito, coerente com a implementação por primitivas da máquina TAM). A divisão `div` é a divisão inteira e `mod` é o resto da divisão inteira.

A chamada de um procedimento executa o corpo do procedimento e retorna ao ponto seguinte à chamada. O uso de uma função em uma expressão executa o corpo da função e o valor da expressão é o último valor atribuído ao nome da função dentro do corpo. As variáveis não são inicializadas automaticamente: o valor de uma variável antes da primeira atribuição é indefinido, e é responsabilidade do programador atribuir antes de usar. Os valores booleanos são representados, em tempo de execução, pelos inteiros 1 (`true`) e 0 (`false`), conforme a convenção da máquina TAM (Seção 9).

2. DESCRIÇÃO GERAL DA ARQUITETURA DO COMPILADOR

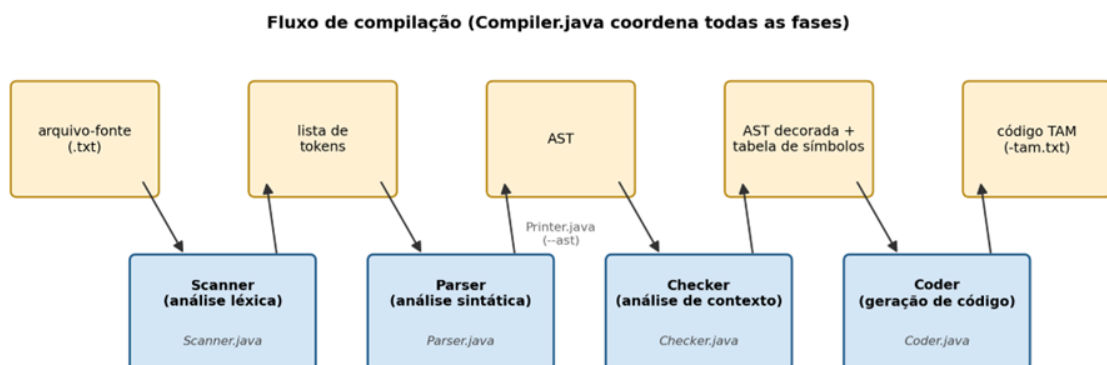
Um compilador é um programa de software essencial no desenvolvimento de sistemas de computação, cuja principal função é transformar código-fonte, escrito em uma linguagem de programação de alto nível, em código de máquina ou código objeto, que pode ser diretamente executado pelo processador do computador. Este processo é fundamental, pois permite que programas escritos em linguagens

compreensíveis pelos seres humanos sejam convertidos para uma forma que as máquinas possam interpretar e executar eficientemente.

O compilador foi implementado na linguagem Java e organizado como uma sequência de fases clássicas, cada uma materializada em uma classe principal. A classe Compiler (com ponto de entrada alternativo Main) coordena o fluxo: lê o arquivo-fonte, aciona as fases solicitadas pelas opções de linha de comando e reporta erros. O fluxo completo é:

```
arquivo-fonte
-> Scanner (análise léxica) -> lista de tokens
-> Parser (análise sintática) -> AST
-> Checker (análise de contexto) -> tabela de símbolos
-> Coder (geração de código) -> arquivo texto com instruções TAM
```

Figura 1 – Arquitetura do compilador: fases e representações intermediárias



Fonte: Os Autores (2026).

Cada fase consome a saída da anterior. O Scanner converte o texto em uma lista de objetos Token (tipo, lexema, linha e coluna). O Parser, um analisador recursivo descendente LL(1), consome a lista de tokens e constrói a árvore de sintaxe abstrata (AST), cujos nós são as classes internas de No. O Checker percorre a AST por meio do padrão de projeto Visitor, constrói a tabela de símbolos (TabelaSimbolos) e verifica as regras de escopo e de tipos. Por fim, o Coder, também um Visitor, percorre novamente a AST e emite as instruções da máquina TAM em um arquivo de texto, apoiado pelo gerador de rótulos (GeradorRotulos). Um

terceiro Visitor, o Printer, imprime a AST no console com indentação, atendendo ao requisito de visualização da árvore.

A tabela a seguir resume a responsabilidade de cada arquivo do diretório src/.

Tabela 3 – Arquivos do compilador e suas responsabilidades

Arquivo	Responsabilidade
Compiler.java	Programa principal: interface de linha de comando, leitura do arquivo-fonte, orquestração das fases e códigos de saída
Main.java	Ponto de entrada alternativo; apenas delega para Compiler
Scanner.java	Analizador léxico: transforma o texto-fonte em tokens, descartando separadores e comentários
Token.java	Definição dos tipos de token, tabela de palavras reservadas e formatação para impressão
Parser.java	Analizador sintático recursivo descendente LL(1); constrói a AST
No.java	Hierarquia de classes (abstratas e concretas) dos nós da AST
Visitor.java	Interface genérica do padrão Visitor para percorrer a AST
Printer.java	Visitor que imprime a AST com indentação (opção --ast)
Checker.java	Visitor da análise de contexto: identificação e verificação de tipos
TabelaSimbolos.java	Tabela de símbolos: nome, categoria, tipo, deslocamento e rótulo de cada identificador
Coder.java	Visitor da geração de código: emite instruções da máquina TAM
GeradorRotulos.java	Gerador de rótulos únicos para desvios e subprogramas
Erro.java / ErroSintatico.java / ErroContexto.java	Exceções de cada fase, com mensagem, linha e coluna

As fases são executadas sob demanda, conforme a opção de linha de comando: sem opção, executa-se apenas a análise léxica; --tokens imprime a lista de tokens; --ast executa léxico e sintático e imprime a árvore; --code executa todas as fases e grava o arquivo de código-objeto. Erros interrompem o processamento da fase corrente com mensagens padronizadas contendo linha e coluna, e o processo

termina com código de saída 1 (erros de compilação) ou 2 (erros de uso/entrada e saída).

Essas etapas colaboram de forma integrada para transformar o código-fonte em um programa executável, garantindo correção, eficiência e robustez no processo de compilação.

3. FUNDAMENTAÇÃO TEÓRICA E TÉCNICAS EMPREGADAS NA ANÁLISE SINTÁTICA

A técnica adotada para a análise sintática foi a análise descendente recursiva (recursive descent), na variante preditiva sem retrocesso, seguindo o modelo do livro-texto adotado na disciplina (WATT, D. A.; BROWN, D. F. Programming Language Processors in Java: Compilers and Interpreters. Prentice Hall, 2000). Nessa técnica, cada símbolo não terminal da gramática dá origem a um método de análise (parsePrograma, parseBloco, parseComando, parseExpressao etc.), e o corpo de cada método espelha as produções do não terminal correspondente. O reconhecimento é dirigido por um token de consulta (lookahead): a cada ponto de decisão, o token corrente determina qual alternativa da produção deve ser seguida, sem necessidade de retrocesso.

Para que essa estratégia funcione de forma determinística, a gramática precisa satisfazer a condição LL(1): para cada não terminal, os conjuntos FIRST das suas alternativas devem ser dois a dois disjuntos e, quando alguma alternativa pode derivar a cadeia vazia, o conjunto FIRST das demais deve ser disjunto do FOLLOW do não terminal. O conjunto $FIRST(\alpha)$ reúne os terminais que podem iniciar alguma cadeia derivada de α ; o conjunto $FOLLOW(A)$ reúne os terminais que podem aparecer imediatamente após A em alguma forma sentencial. Gramáticas com recursão à esquerda ou com alternativas de prefixo comum violam a condição e precisam ser transformadas: a recursão à esquerda é eliminada convertendo produções recursivas em repetições (laços, na notação EBNF), e prefixos comuns são resolvidos por fatoração à esquerda. As transformações aplicadas neste projeto e a demonstração da condição LL(1) são apresentadas nas Seções 4 e 5.

O método consumir(tipo) (correspondente ao accept/match da literatura) confere se o token corrente é do tipo esperado: em caso positivo, avança; em caso negativo, lança ErroSintatico com linha, coluna, o que era esperado e o que foi

encontrado. A construção da AST é integrada aos métodos de análise: cada método retorna o nó correspondente ao trecho reconhecido, de modo que a árvore é montada durante o próprio fluxo de reconhecimento, sem passagem adicional. A estratégia de tratamento de erros é a de modo pânico simplificado: o primeiro erro interrompe a análise, o que é adequado ao propósito didático do projeto e garante mensagens precisas.

Também fundamentam o projeto: a teoria de linguagens regulares para a fase léxica (tokens especificados por expressões regulares e reconhecidos por um autômato finito determinístico codificado manualmente, com a política do maior casamento possível, maximal munch); o padrão de projeto Visitor para separar a estrutura da AST das operações realizadas sobre ela (impressão, verificação de contexto e geração de código); e o modelo de máquina de pilha da TAM (Triangle Abstract Machine) para a geração de código, com avaliação de expressões em notação pós-fixa e rotinas primitivas para os operadores.

4. ANÁLISE LÉXICA

A análise léxica é realizada pela classe Scanner, que percorre o texto-fonte caractere a caractere e produz a sequência de tokens consumida pelo analisador sintático. Cada token carrega quatro informações: o tipo (um valor do enum Token.Tipo), o lexema (o texto reconhecido), a linha e a coluna em que inicia — informações usadas em todas as mensagens de erro do compilador.

4.1. Relação de tokens

A partir da gramática fornecida, foi elaborada a relação completa dos tokens da linguagem, dividida em classes: tokens de estrutura, palavras reservadas, operadores e pontuação. As palavras reservadas são reconhecidas sem distinção entre maiúsculas e minúsculas (Program, PROGRAM e program produzem o mesmo token).

Tabela 4 – Relação de tokens da linguagem

Token (enum)	Lexema(s)	Descrição
IDENTIFICADOR	soma, _x1, total	Nome definido pelo programador
LITERAL_INTEIRO	0, 42, 750	Constante numérica inteira
LITERAL_TEXTO	'ola'	Texto entre aspas simples (reconhecido no léxico; rejeitado na análise de contexto)
FIM_ARQUIVO	(vazio)	Marcador de fim da entrada
PROGRAMA	program	Início do programa
VAR	var	Início da seção de variáveis
PROCEDIMENTO	procedure	Declaração de procedimento
FUNCAO	function	Declaração de função
INICIO / FIM	begin / end	Delimitadores de comando composto
SE / ENTAO / SENAO	if / then / else	Comando condicional
ENQUANTO / FACA	while / do	Comando de repetição
INTEIRO / BOOLEANO	integer / boolean	Denotadores de tipo
VERDADEIRO / FALSO	true / false	Literais booleanos
E / OU / NAO	and / or / not	Operadores lógicos
DIV / MOD	div / mod	Divisão inteira e resto
MAIS / MENOS / VEZES / BARRA	+ - * /	Operadores aritméticos (ver observação sobre '/')
ATRIBUICAO	:=	Atribuição
IGUAL / DIFERENTE	= <>	Igualdade e desigualdade
MENOR / MENOR_IGUAL	< <=	Relacionais
MAIOR / MAIOR_IGUAL	> >=	Relacionais
PONTO_E_VIRGULA	;	Separador de comandos e declarações
DOIS_PONTOS	:	Separador de tipo na declaração
VIRGULA	,	Separador de lista de identificadores
PONTO	.	Fim de programa
ABRE_PARENTESE FECHA_PARENTESE	/ ()	Agrupamento de expressões
ABRE_COLCHETE FECHA_COLCHETE	/ []	Reservados para extensões futuras (não usados pela gramática sintática)

Duas observações registradas pela equipe: (i) o token BARRA (/) é reconhecido pelo léxico, mas a gramática sintática usa exclusivamente div para a

divisão inteira — o uso de '/' em uma expressão produz, portanto, erro sintático; (ii) os colchetes são tokenizados por completude, mas nenhuma produção da gramática sintática os utiliza.

4.2. Gramática léxica

A gramática léxica descreve, para cada classe de token, como os seus lexemas são formados a partir dos caracteres da entrada. Trata-se de uma gramática regular, apresentada abaixo em EBNF:

```
token          ::= identificador | literal-inteiro | literal-texto
                | operador | pontuacao
identificador::= (letra | '_' ) (letra | digito | '_' )*
literal-inteiro ::= digito digito*
literal-texto::= "" caractere-sem-aspa-e-sem-quebra* ""
operador       ::= '+' | '-' | '*' | '/' | ':' | '=' | '<' | '>'
                | '<=' | '>=' | '<>'
pontuacao      ::= ';' | ':' | ',' | '.' | '(' | ')' | '[' | ']'

separador      ::= espaco | tabulacao | quebra-de-linha | comentario
comentario     ::= '{' caractere-sem-chave* '}'
                | '(*' qualquer-texto-sem-fecha* '*)'
                | '//' caractere-sem-quebra*

letra          ::= 'a' | ... | 'z' | 'A' | ... | 'Z'
digito         ::= '0' | ... | '9'
```

As palavras reservadas (program, var, procedure, function, begin, end, if, then, else, while, do, integer, boolean, true, false, and, or, not, div, mod) têm a mesma forma de um identificador; a distinção é feita por consulta a uma tabela após o reconhecimento, técnica descrita na Seção 4.5. De forma equivalente, o conjunto de tokens pode ser descrito por expressões regulares:

Tabela 5 – Expressões regulares dos tokens e separadores

Classe	Expressão regular
Identificador / palavra reservada	<code>[A-Za-z_][A-Za-z0-9_]*</code>
Líteral inteiro	<code>[0-9]+</code>
Líteral de texto	<code>'[^\\n\\r]*'</code>
Operadores e pontuação	<code>\+ - * / := = <> <= < >= > ; : , \. \ (\) \ [\]</code>
Comentários	<code>\{ [^}]* \} \ (* ([^*] * [^]) * *) // [^\\n]*</code>
Espaços	<code>[\t\r\n]+</code>

4.3. Manipulações efetuadas na gramática original e justificativas

Sobre o conjunto léxico derivado da gramática fornecida foram efetuadas as seguintes manipulações e decisões, todas com o objetivo de tornar o reconhecimento determinístico e melhorar a experiência de uso:

Separação entre gramática léxica e sintática. Os símbolos terminais da gramática original foram extraídos para uma gramática regular própria (Seção 4.2), de modo que o analisador sintático trabalhe sobre tokens, e não sobre caracteres. Essa é a manipulação clássica que viabiliza a arquitetura em duas fases.

Palavras reservadas unificadas com identificadores. Em vez de uma produção léxica para cada palavra-chave (o que criaria ambiguidade com a produção de identificador, pois toda palavra-chave também casa com o padrão de identificador), o léxico reconhece um identificador genérico e depois consulta a tabela de palavras reservadas. Isso elimina o conflito e simplifica o autômato.

Fatoração dos operadores compostos. Os pares com prefixo comum — ':' e ':='; '<', '<=' e '<>'; '>' e '>=' — foram fatorados: após ler o primeiro caractere, o Scanner examina o caractere seguinte para decidir o token (política do maior casamento possível). O mesmo vale para distinguir '(' de '('*' (abre-parêntese versus início de comentário) e '/' de '/' (operador versus comentário de linha).

Comentários e espaços tratados como separadores descartáveis. Foram movidos para fora da gramática de tokens e consumidos em um laço prévio ao reconhecimento de cada token, simplificando as produções.

Restrição de justaposição número–letra. Decidiu-se rejeitar, ainda no léxico, um número imediatamente seguido de letra ou '_' (por exemplo, 2abc), que de outro modo seria tokenizado como LITERAL_INTEIRO seguido de IDENTIFICADOR e produziria um erro sintático confuso mais adiante. A mensagem léxica resultante é mais clara para o usuário.

4.4. Verificação da condição LL(1) na gramática manipulada

Para a gramática léxica, a condição LL(1) traduz-se em: o primeiro caractere da entrada (após o descarte de separadores) deve determinar univocamente a

classe de token a reconhecer. Verifica-se que os conjuntos FIRST das alternativas de token são dois a dois disjuntos:

Tabela 6 – Conjuntos FIRST das produções léxicas

Produção	FIRST (primeiro caractere)
identificador / palavra reservada	letra, '_'
literal-inteiro	dígito
literal-texto	' (aspa simples)
operadores e pontuação	+ - * / ; : , . () [] = < >
separadores	espaço, tab, CR, LF, '{', '(*', '/'

Como letra/underscore, dígito, aspa simples e cada símbolo especial iniciam classes distintas, não há interseção entre os conjuntos. Os únicos pontos em que um caractere não decide sozinho — ':' (DOIS_PONTOS ou ATRIBUICAO), '<' (MENOR, MENOR_IGUAL ou DIFERENTE), '>' (MAIOR ou MAIOR_IGUAL), '(' (FECHA_PARENTESE ou comentário '(*') e '/' (BARRA ou comentário '//') — foram resolvidos pela fatoração descrita na Seção 4.3, que reduz a decisão à consulta de um único caractere adicional. Após a fatoração, cada decisão é tomada com no máximo um caractere de antecipação, o que corresponde exatamente à condição LL(1) no nível léxico.

4.5. Técnicas utilizadas e visão geral do funcionamento

O Scanner implementa manualmente um autômato finito determinístico, seguindo o modelo do livro-texto. Os elementos centrais são:

Cursor com posição de linha e coluna. Os métodos peek() (consulta o caractere corrente sem consumir), peekNext() (consulta o seguinte) e advance() (consome e atualiza linha/coluna) encapsulam o percurso do texto. A contagem de linha e coluna permite mensagens de erro precisas em todas as fases.

Descarte de separadores. Antes de cada token, skipSeparatorsAndComments() consome, em laço, espaços em branco e comentários das três formas, inclusive combinações consecutivas. Comentário aberto e não fechado gera erro léxico imediato, apontando a linha e a coluna em que o comentário começou.

Despacho pelo primeiro caractere. O método `nextToken()` decide a classe do token pelo primeiro caractere: letra ou `'_'` leva a `readIdentifier()`; dígito, a `readNumber()`; aspa simples, a `readString()`; qualquer outro, a `readSymbol()`, que trata os símbolos simples e compostos com o auxílio de `match()` (consome o próximo caractere apenas se for o esperado — maximal munch).

Tabela de palavras reservadas. Reconhecido um lexema de identificador, `Token.palavraReservadaOuIdentificador()` consulta um `Map` imutável (em minúsculas, garantindo a insensibilidade a caixa) e devolve o tipo de palavra reservada correspondente, ou `IDENTIFICADOR` quando o lexema não consta na tabela.

Interface com o Parser. O método `scanAll()` produz a lista completa de tokens, sempre encerrada pelo token `FIM_ARQUIVO`, que o Parser usa para confirmar que não há texto após o ponto final do programa .

4.6. Exemplos de entradas e saídas

4.6.1. Aceitação de cadeia bem formada

Para o programa exemplos/valido2.txt:

```
program Valido;
var
  x, y: integer;
  positivo: boolean;
begin
  x := 10;
  y := x div 2 + 3;
  positivo := y >= 0;
  if positivo then
    y := y + 1
  else
    y := y - 1;
  while y > 0 do
    y := y - 1
end.
```

o comando `java -cp bin Compiler exemplos\valido2.txt --tokens` produz a saída abaixo:

Figura 2 – Saída da análise léxica (--tokens) para exemplos/valido2.txt

```
PS D:\-UNIVASF-Compilador-Pascal-TAM> java -cp bin Compiler exemplos\valido2.txt --tokens
PROGRAMA      "program"      linha 1, coluna 1
IDENTIFICADOR "Valido"      linha 1, coluna 9
PONTO_E_VIRGULA ";"      linha 1, coluna 15
VAR           "var"      linha 2, coluna 1
IDENTIFICADOR "x"      linha 3, coluna 3
VIRGULA      ","      linha 3, coluna 4
IDENTIFICADOR "y"      linha 3, coluna 6
DOIS_PONTOS  ":"      linha 3, coluna 7
INTEIRO      "integer"  linha 3, coluna 9
PONTO_E_VIRGULA ";"      linha 3, coluna 16
IDENTIFICADOR "positivo" linha 4, coluna 3
DOIS_PONTOS  ":"      linha 4, coluna 11
BOOLEANO     "boolean"  linha 4, coluna 13
PONTO_E_VIRGULA ";"      linha 4, coluna 20
INICIO       "begin"    linha 5, coluna 1
IDENTIFICADOR "x"      linha 6, coluna 3
ATRIBUICAO   ":@"      linha 6, coluna 5
LITERAL_INTEIRO "10"      linha 6, coluna 8
PONTO_E_VIRGULA ";"      linha 6, coluna 10
IDENTIFICADOR "y"      linha 7, coluna 3
ATRIBUICAO   ":@"      linha 7, coluna 5
IDENTIFICADOR "x"      linha 7, coluna 8
DIV          "div"      linha 7, coluna 10
LITERAL_INTEIRO "2"      linha 7, coluna 14
MAIS         "+"      linha 7, coluna 16
LITERAL_INTEIRO "3"      linha 7, coluna 18
PONTO_E_VIRGULA ";"      linha 7, coluna 19
IDENTIFICADOR "positivo" linha 8, coluna 3
ATRIBUICAO   ":@"      linha 8, coluna 12
IDENTIFICADOR "y"      linha 8, coluna 15
MAIOR_IGUAL  ">="      linha 8, coluna 17
LITERAL_INTEIRO "0"      linha 8, coluna 20
PONTO_E_VIRGULA ";"      linha 8, coluna 21
SE           "if"      linha 9, coluna 3
IDENTIFICADOR "positivo" linha 9, coluna 6
ENTAO        "then"    linha 9, coluna 15
IDENTIFICADOR "y"      linha 10, coluna 5
ATRIBUICAO   ":@"      linha 10, coluna 7
IDENTIFICADOR "y"      linha 10, coluna 10
MAIS         "+"      linha 10, coluna 12
LITERAL_INTEIRO "1"      linha 10, coluna 14
SENAO        "else"    linha 11, coluna 3
IDENTIFICADOR "y"      linha 12, coluna 5
ATRIBUICAO   ":@"      linha 12, coluna 7
IDENTIFICADOR "y"      linha 12, coluna 10
MENOS        "-"      linha 12, coluna 12
LITERAL_INTEIRO "1"      linha 12, coluna 14
PONTO_E_VIRGULA ";"      linha 12, coluna 15
ENQUANTO     "while"   linha 13, coluna 3
IDENTIFICADOR "y"      linha 13, coluna 9
MAIOR        ">"      linha 13, coluna 11
LITERAL_INTEIRO "0"      linha 13, coluna 13
FACA         "do"      linha 13, coluna 15
IDENTIFICADOR "y"      linha 14, coluna 5
ATRIBUICAO   ":@"      linha 14, coluna 7
IDENTIFICADOR "y"      linha 14, coluna 10
MENOS        "-"      linha 14, coluna 12
LITERAL_INTEIRO "1"      linha 14, coluna 14
FIM          "end"      linha 15, coluna 1
PONTO        "."      linha 15, coluna 4
FIM_ARQUIVO  ""      linha 16, coluna 1
```

4.6.2. Rejeição de cadeias mal formadas

Para o arquivo exemplos/erro-lexico.txt, que contém o caractere '@' (inexistente no alfabeto da linguagem):

```
program ErroLexico;  
begin  
  x := 10 @ 2  
end.
```

a execução com --tokens termina com código de saída 1 e a mensagem:

```
Erro lexico na linha 3, coluna 11: caractere inesperado '@'
```

Para o arquivo exemplos/erro-lexico-comentario.txt, com comentário de chaves aberto e nunca fechado:

```
program ComentarioAberto;  
begin  
  { comentario sem fechamento  
  x := 1  
end.
```

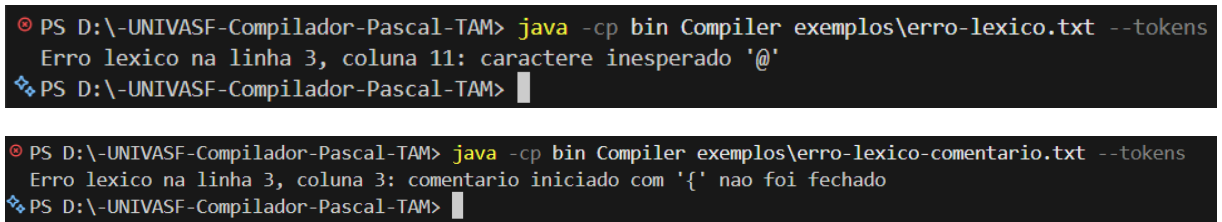
a saída é:

```
Erro lexico na linha 3, coluna 3: comentario iniciado com '{' nao foi fechado
```

E para um número imediatamente seguido de letra (x := 2abc), o léxico rejeita com:

```
Erro lexico na linha 4, coluna 9: numero inteiro nao pode ser seguido diretamente por letra ou '_'
```

Figura 3 – Mensagens de erro léxico no terminal



```
PS D:\-UNIVASF-Compilador-Pascal-TAM> java -cp bin Compiler exemplos\erro-lexico.txt --tokens  
Erro lexico na linha 3, coluna 11: caractere inesperado '@'  
PS D:\-UNIVASF-Compilador-Pascal-TAM>  
  
PS D:\-UNIVASF-Compilador-Pascal-TAM> java -cp bin Compiler exemplos\erro-lexico-comentario.txt --tokens  
Erro lexico na linha 3, coluna 3: comentario iniciado com '{' nao foi fechado  
PS D:\-UNIVASF-Compilador-Pascal-TAM>
```

5. ANÁLISE SINTÁTICA

A análise sintática é realizada pela classe Parser, um analisador recursivo descendente preditivo que consome a lista de tokens do Scanner e constrói a AST. Esta seção apresenta a gramática sintática implementada, as transformações aplicadas sobre a gramática original, a demonstração da condição LL(1) por meio dos conjuntos FIRST e FOLLOW e exemplos reais de aceitação e rejeição.

5.1. Gramática sintática

A gramática sintática efetivamente implementada é a apresentada na Seção 1.2, reproduzida aqui de forma compacta. Cada não terminal corresponde a um método do Parser.

```
Programa ::= 'program' IDENT ';' Bloco '.'
Bloco ::= SecaoVar? Subprograma* CompostoCmd
SecaoVar ::= 'var' DeclVar+
DeclVar ::= ListIdent ':' Tipo ';'
ListIdent ::= IDENT (',' IDENT)*
Tipo ::= 'integer' | 'boolean'
Subprograma ::= 'procedure' IDENT ';' Bloco ';' | 'function' IDENT ':' Tipo ';' Bloco ';'
CompostoCmd ::= 'begin' (Cmd (',' Cmd)*)? 'end'
Cmd ::= IDENT CmdIdent | SeCmd | EnquantoCmd | CompostoCmd | vazio
CmdIdent ::= '=' Expr | vazio (fatoração: atribuição × chamada)
SeCmd ::= 'if' Expr 'then' Cmd ('else' Cmd)?
EnquantoCmd ::= 'while' Expr 'do' Cmd
Expr ::= ExpSimples (OpRel ExpSimples)?
OpRel ::= '=' | '<>' | '<' | '<=' | '>' | '>='
ExpSimples ::= Termo (('+' | '-' | 'or') Termo)*
Termo ::= Fator (('*' | 'div' | 'mod' | 'and') Fator)*
Fator ::= IDENT | LITERAL_INTEIRO | LITERAL_TEXTO | 'true' | 'false' | '(' Expr ')'
| 'not' Fator | '-' Fator
```

5.2. Manipulações efetuadas na gramática original e justificativas

A gramática fornecida, na sua forma original em BNF, não era diretamente adequada à análise descendente. As seguintes transformações foram aplicadas

para obter uma gramática equivalente (que gera exatamente a mesma linguagem) e adequada ao método LL(1):

- **Eliminação de recursão à esquerda nas expressões.** Produções na forma clássica $\text{ExpSimples} ::= \text{ExpSimples} \text{'+'} \text{Termo} \mid \text{Termo}$ são recursivas à esquerda e fazem um analisador descendente entrar em laço infinito. Foram reescritas na forma iterativa (EBNF) $\text{ExpSimples} ::= \text{Termo} ((\text{'+'} \mid \text{'-' } \mid \text{'or'}) \text{Termo})^*$, implementada com um laço while no método de análise. O mesmo foi feito em Termo. A forma iterativa preserva a associatividade à esquerda dos operadores, pois o laço constrói a árvore acumulando o operando esquerdo.
- **Fatoração à esquerda em Cmd.** As alternativas $\text{AtribCmd} ::= \text{IDENT} \text{'='} \text{Expr}$ e $\text{ChamadaProc} ::= \text{IDENT}$ começam ambas com IDENT, criando conflito FIRST/FIRST. A gramática foi fatorada com o não terminal auxiliar CmdIdent (Seção 5.1): após consumir o IDENT, o token seguinte decide — '=' indica atribuição; qualquer outro token de $\text{FOLLOW}(\text{Cmd})$ indica chamada de procedimento. Na implementação, essa fatoração aparece como uma consulta ao segundo token (método $\text{proximo}()$) antes de escolher o método de análise, o que é operacionalmente equivalente.
- **Resolução do dangling else.** A produção $\text{SeCmd} ::= \text{'if' Expr 'then' Cmd ('else' Cmd)?}$ é ambígua na presença de ifs aninhados: o else poderia associar-se a qualquer if aberto. Adotou-se a convenção universal de associá-lo ao if mais próximo, materializada naturalmente no método $\text{parseSe}()$: ao encontrar else, ele o consome para o if que está sendo analisado no momento.
- **Comando vazio.** A alternativa vazio em Cmd permite ponto-e-vírgula supérfluo antes de end (padrão comum em Pascal) e sequências como begin end. Sem ela, programas idiomáticos seriam rejeitados.
- **Estratificação de expressões por precedência.** A hierarquia $\text{Expr} \rightarrow \text{ExpSimples} \rightarrow \text{Termo} \rightarrow \text{Fator}$ codifica a precedência dos operadores diretamente na gramática, dispensando qualquer tabela de precedência no código.

5.3. Verificação da condição LL(1) na gramática manipulada

Uma gramática é LL(1) quando, em todo ponto de decisão, o próximo token determina univocamente a alternativa a seguir. Demonstra-se a propriedade pelos conjuntos FIRST e FOLLOW da gramática da Seção 5.1.

Tabela 7 – Conjuntos FIRST dos não terminais

Não terminal	FIRST
Programa	program
Bloco	var, procedure, function, begin
SecaoVar	var
DeclVar / ListIdent	IDENT
Tipo	integer, boolean
Subprograma	procedure, function
CompostoCmd	begin
Cmd	IDENT, if, while, begin, ϵ
CmdIdent	:=, ϵ
SeCmd	if
EnquantoCmd	while
Expr / ExpSimples / Termo / Fator	IDENT, LITERAL_INTEIRO, LITERAL_TEXTO, true, false, (, not, -
OpRel	=, <>, <, <=, >, >=

Tabela 8 – Conjuntos FOLLOW relevantes

Não terminal	FOLLOW
Bloco	. ;
Cmd	; end else (fim de arquivo)
CmdIdent	; end else (fim de arquivo)
Expr	then do ; end else) .
ExpSimples	=, <>, <, <=, >, >= $\cup$ FOLLOW(Expr)
Termo	+, -, or $\cup$ FOLLOW(ExpSimples)
Fator	*, div, mod, and $\cup$ FOLLOW(Termo)
Tipo	;
SecaoVar	procedure, function, begin

Com esses conjuntos, verificam-se as condições em cada ponto de decisão da gramática:

- **Cmd:** as alternativas iniciam por IDENT, if, while e begin — conjuntos unitários e disjuntos. A alternativa vazia não conflita: $\text{FIRST}(\text{Cmd}) \cap \text{FOLLOW}(\text{Cmd}) = \{\text{IDENT, if, while, begin}\} \cap \{;, \text{end, else, eof}\} = \emptyset$.
- **CmdIdent:** $\text{FIRST}(':= \text{Expr}) = \{:=\}$ e $\text{FOLLOW}(\text{CmdIdent}) = \{;, \text{end, else, eof}\}$; interseção vazia — a fatoração eliminou o conflito original entre atribuição e chamada.
- **Fator:** as oito alternativas iniciam por tokens distintos (IDENT, literal inteiro, literal de texto, true, false, '(', not, '-'); conjuntos unitários e disjuntos.
- **Bloco:** SecaoVar? decide-se por $\text{var} \notin \text{FIRST}(\text{Subprograma}) \cup \{\text{begin}\}$; a repetição Subprograma^* decide-se por $\text{procedure/function} \notin \{\text{begin}\}$; ambas sem conflito FIRST/FOLLOW.
- **Expressões:** a parte opcional $(\text{OpRel ExpSimples})?$ em Expr decide-se porque $\text{FIRST}(\text{OpRel}) \cap \text{FOLLOW}(\text{Expr}) = \emptyset$; as repetições em ExpSimples e Termo decidem-se porque $\{+, -, \text{or}\}$ e $\{*, \text{div, mod, and}\}$ são disjuntos dos respectivos FOLLOWS.
- **SeCmd (caso limite):** na parte $(\text{'else' Cmd})?$, o token else pertence tanto a FIRST da parte opcional quanto a FOLLOW(Cmd) — é o único conflito FIRST/FOLLOW remanescente, inerente ao dangling else de qualquer gramática Pascal-like. Ele é resolvido deterministicamente pela convenção “else liga ao if mais próximo”: o analisador sempre prefere consumir o else. Com essa regra de desempate documentada, a gramática é processada por um analisador preditivo com um token de antecipação, sem retrocesso, em todos os casos.

Conclui-se que a gramática original não era LL(1) (recursão à esquerda nas expressões e prefixo comum IDENT em Cmd), e que a gramática manipulada satisfaz a condição LL(1) em todos os pontos de decisão, com a única exceção clássica do else opcional, resolvida por convenção, exatamente como no livro-texto e nos compiladores Pascal reais.

5.4. Técnicas utilizadas e visão geral do funcionamento

O Parser implementa o método recursivo descendente com um método por não terminal: `parsePrograma`, `parseBloco`, `parseSecaoVar`, `parseDeclVar`, `parseTipo`, `parseSubprograma`, `parseComposto`, `parseComando`, `parseAtribuicao`, `parseSe`, `parseEnquanto`, `parseExpressao`, `parseExpSimples`, `parseTermo` e `parseFator`. O estado do analisador é a lista de tokens e um índice de posição; os métodos utilitários `atual()` (token corrente), `proximo()` (segundo token, usado apenas na decisão atribuição × chamada), `avancar()` e `consumir(tipo)` encapsulam o consumo.

O método `consumir(tipo)` é o ponto central do tratamento de erros: se o token corrente não é do tipo esperado, lança `ErroSintatico` com a linha e a coluna do token encontrado e uma mensagem no formato “esperava 'X', encontrado 'Y'”, em que X é o nome amigável do token (por exemplo, ';' em vez de `PONTO_E_VIRGULA`). Além do fluxo normal, há verificações específicas com mensagens dedicadas: seção var sem nenhuma declaração, denotador de tipo inválido e expressão esperada.

A construção da AST é simultânea ao reconhecimento: cada método de análise retorna o nó correspondente (por exemplo, `parseSe` retorna um `No.Se` com condição, ramo então e ramo senão), de modo que, ao final de `parsePrograma`, a árvore completa está montada. Após o ponto final do programa, o Parser exige o token `FIM_ARQUIVO`, garantindo que não exista texto residual após o fim do programa.

5.5. Exemplos de entradas e saídas

5.5.1 Aceitação de cadeia bem formada

O comando `java -cp bin Compiler exemplos\valido2.txt --ast` aceita o programa da Seção 4.6.1 e imprime a AST (a saída completa é apresentada na Seção 7.2). A ausência de mensagens de erro e o código de saída 0 comprovam a aceitação. O programa `exemplos/procedimento-funcao.txt`, com procedimento e função, é igualmente aceito.

5.5.2 Rejeição de cadeia mal formada

O arquivo exemplos/erro-sintatico.txt omite o ponto-e-vírgula após o cabeçalho do programa:

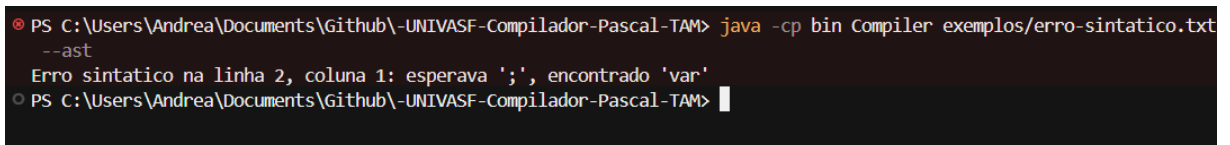
```
program ErroSintatico
var
  x: integer;
begin
  x := 1;
end.
```

A execução com --ast rejeita a cadeia com código de saída 1 e a mensagem (execução real):

```
Erro sintatico na linha 2, coluna 1: esperava ';', encontrado 'var'
```

Outros exemplos de rejeição observados nos testes: 'if x 1 then ...' produz “esperava 'then', encontrado '1'”; 'x := ;' produz “expressao esperada, encontrado ';'”; e 'var x: real;' produz “tipo esperado ('integer' ou 'boolean'), encontrado 'real'”.

Figura 4 – Mensagem de erro sintático no terminal

A terminal window with a dark background. The prompt is 'PS C:\Users\Andrea\Documents\Github\UNIVASF-Compilador-Pascal-TAM>'. The user enters 'java -cp bin Compiler exemplos/erro-sintatico.txt --ast'. The output is 'Erro sintatico na linha 2, coluna 1: esperava ';', encontrado 'var''. The prompt returns.

```
PS C:\Users\Andrea\Documents\Github\UNIVASF-Compilador-Pascal-TAM> java -cp bin Compiler exemplos/erro-sintatico.txt
--ast
Erro sintatico na linha 2, coluna 1: esperava ';', encontrado 'var'
PS C:\Users\Andrea\Documents\Github\UNIVASF-Compilador-Pascal-TAM>
```

6. DESCRIÇÃO DAS PRINCIPAIS ESTRUTURAS DE DADOS UTILIZADAS

Para gerenciar as etapas do compilador estruturas de dados são necessárias, as utilizadas no compilador foram:

- **Token e List<Token>.** Cada token é um objeto imutável com quatro campos públicos finais: tipo (enum Token.Tipo, com os 40 valores da Tabela 4), lexema (String), linha e coluna (int). O Scanner produz um ArrayList<Token> encerrado por FIM_ARQUIVO; o Parser navega nessa lista por índice, o que torna trivial a consulta ao token seguinte (lookahead).
- **AST — hierarquia de nós (No.java).** A árvore de sintaxe abstrata é formada por classes internas da classe abstrata No, organizadas em três subfamílias: declarações (DeclaracaoVar, e as subclasses de

DeclaracaoSubprograma: DeclaracaoProcedimento e DeclaracaoFuncao), comandos (subclasses da abstrata Comando: Composto, Atribuicao, ChamadaProcedimento, Se, Enquanto e Vazio) e expressões (subclasses da abstrata Expressao: Binario, Unario, Identificador, LiteralInteiro, LiteralBooleano e LiteralTexto). Os nós Programa e Bloco completam a estrutura. Todos os campos são finais e públicos, e cada nó guarda os tokens relevantes (nomes, operadores, palavras-chave), preservando linha e coluna para mensagens de erro das fases posteriores. O nó Vazio é um singleton (No.Vazio.INSTANCIA), já que não carrega estado.

- **Visitor<T>**. Interface genérica com um método visitarX para cada classe concreta de nó; cada nó implementa aceitar(Visitor<T>), que despacha para o método correto (double dispatch). O parâmetro de tipo T permite reutilizar a mesma interface com propósitos distintos: o Printer usa T = Void (apenas efeito de impressão), o Checker usa T = TabelaSimbolos.Tipo (cada expressão retorna o seu tipo) e o Coder usa T = Void (efeito de emissão de instruções).
- **TabelaSimbolos**. Implementada sobre um LinkedHashMap<String, Entrada>, que preserva a ordem de declaração — propriedade usada pela geração de código para emitir a instrução PUSH com as variáveis na ordem em que foram declaradas. Cada Entrada registra: nome, categoria (VARIABEL, PROCEDIMENTO ou FUNCAO), tipo (INTEIRO ou BOOLEANO; nulo para procedimentos), deslocamento (posição de alocação na pilha, atribuída sequencialmente às variáveis) e rótulo (rótulo de desvio do subprograma; nulo para variáveis). Os métodos de consulta especializados (buscar, buscarVariavelOuFuncao e buscarProcedimento) embutem as verificações de categoria e lançam ErroContexto com mensagens específicas.
- **Lista de instruções e GeradorRotulos**. O código-objeto é acumulado em um ArrayList<String>, uma instrução por posição. O GeradorRotulos mantém um contador inteiro e produz rótulos únicos concatenando um prefixo descritivo ao contador (MAIN_0, ELSE_1, FIM_IF_2, WHILE_3,

FIM_WHILE_4, PROC_incrementar_0, FUNC_positivo_1), o que dispensa retrocorreção (backpatching) de endereços numéricos.

- **Exceções por fase.** Erro (léxico), ErroSintatico e ErroContexto são exceções não verificadas com linha e coluna; a classe Compiler as captura por fase e as converte em mensagens no fluxo de erro padrão (stderr) e em códigos de saída apropriados.

7. MONTAGEM E IMPRESSÃO DA ÁRVORE DE SINTAXE ABSTRATA

7.1. Estruturas de dados e algoritmos utilizados

A AST é representada pela hierarquia de classes descrita na Seção 6: a classe abstrata No define o método abstrato aceitar(Visitor<T>), e cada construção da linguagem possui uma classe concreta de nó. As classes abstratas intermediárias (Comando, Expressao e DeclaracaoSubprograma) agrupam as famílias de nós e permitem que os campos dos nós pais sejam tipados com precisão (por exemplo, a condição de um No.Se é uma Expressao; o corpo, um Comando). A árvore é abstrata no sentido próprio: pontuação, palavras-chave estruturais e parênteses de agrupamento não geram nós — parênteses, em particular, desaparecem, pois a estrutura da árvore já registra a ordem de avaliação.

A montagem ocorre durante a análise sintática (síntese dirigida pela sintaxe): cada método parseX do Parser retorna o nó do trecho reconhecido, e os métodos de nível superior compõem esses retornos nos seus próprios nós. Não há passagem separada de construção.

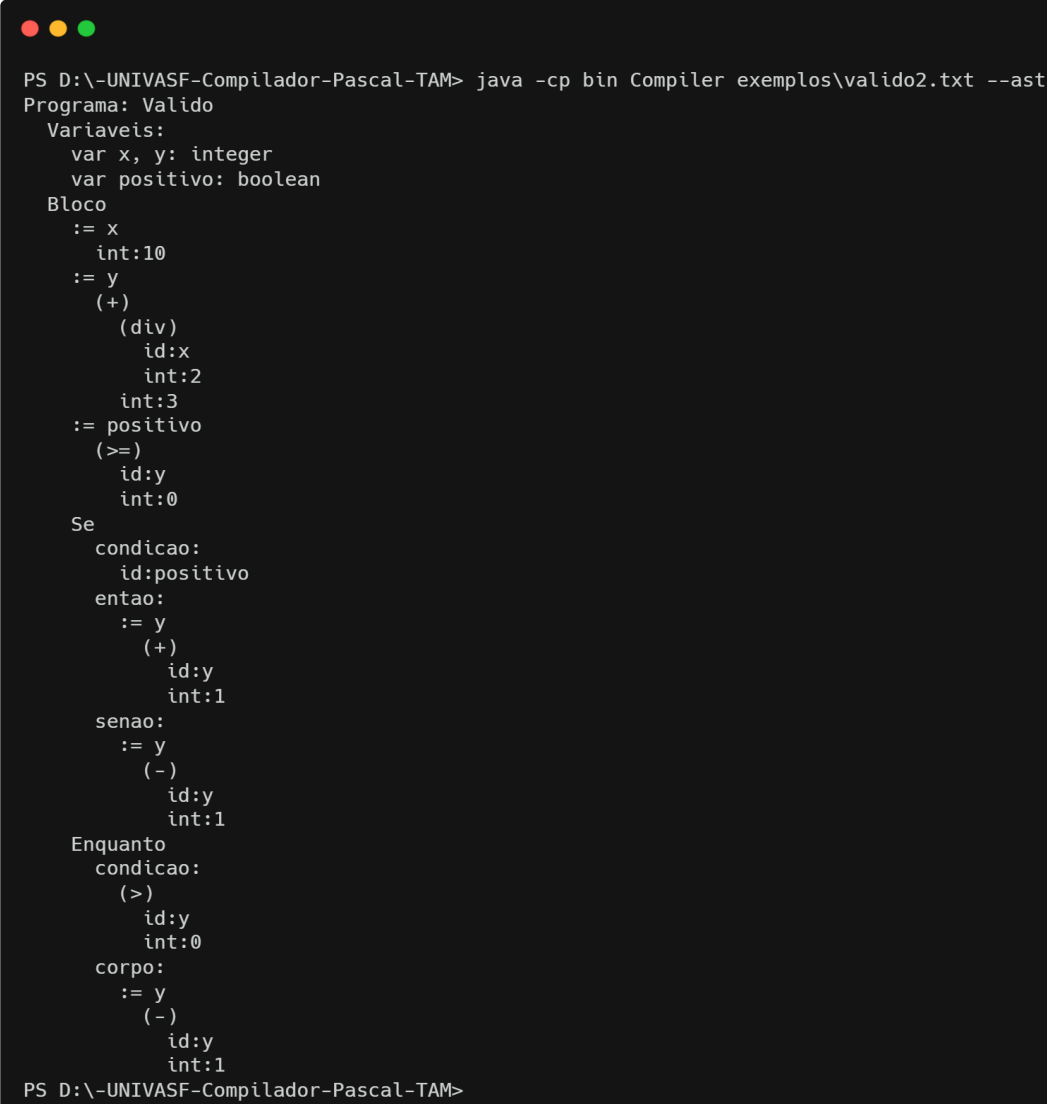
A visualização é feita pela classe Printer, um Visitor<Void> que percorre a árvore em profundidade (pré-ordem) mantendo um contador de nível de indentação: ao visitar um nó, imprime uma linha descritiva com recuo de dois espaços por nível, incrementa o nível, visita os filhos e decrementa. As convenções de notação são: “:= variável” para atribuição, “(op)” para operador binário, “[op]” para operador unário, “id:nome”, “int:valor”, “bool:valor” para folhas, “call nome” para chamada de procedimento e linhas “condicao:”, “entao:”, “senao:” e “corpo:” para as partes de if e while. O usuário aciona a impressão com a opção --ast.

O padrão Visitor, exigido pelo enunciado, é o mecanismo que permite que impressão (Printer), verificação de contexto (Checker) e geração de código (Coder) sejam três operações completamente separadas da estrutura dos nós: adicionar uma nova operação sobre a árvore significa criar uma nova classe que implementa Visitor, sem alterar nenhuma classe de nó.

7.2. Exemplos de programas-fonte e respectivas árvores geradas

Para o programa exemplos/valido2.txt (Seção 4.6.1), o comando `java -cp bin Compiler exemplos\valido2.txt --ast` imprime

Figura 5 – Árvore sintática abstrata gerada para o arquivo valido2.txt

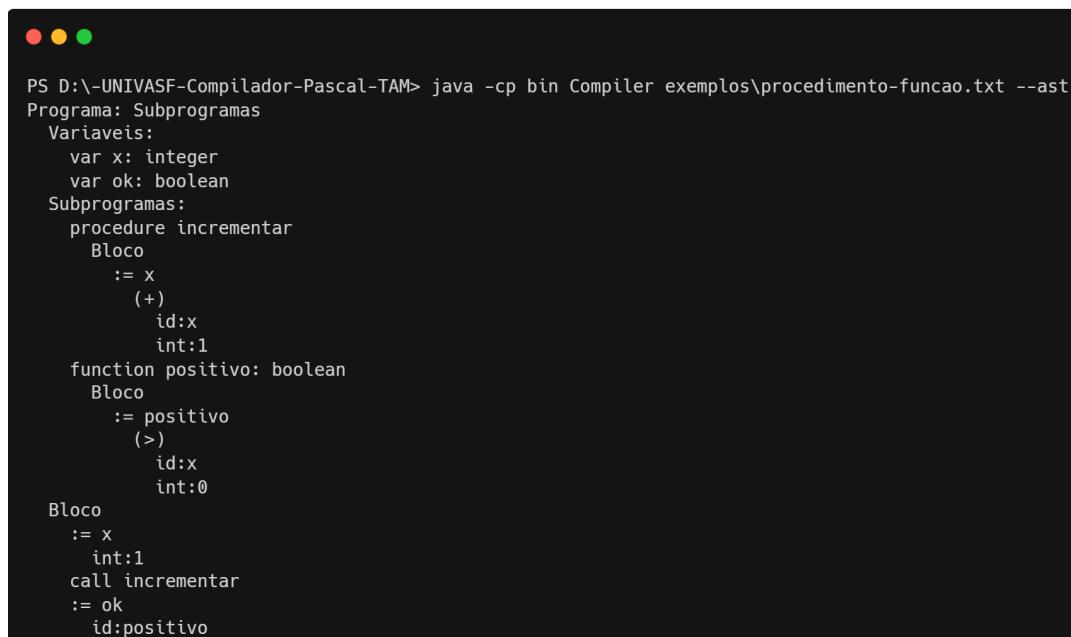


```
PS D:\-UNIVASF-Compilador-Pascal-TAM> java -cp bin Compiler exemplos\valido2.txt --ast
Programa: Valido
Variaveis:
  var x, y: integer
  var positivo: boolean
Bloco
  := x
    int:10
  := y
    (+)
      (div)
        id:x
        int:2
      int:3
  := positivo
    (>=)
      id:y
      int:0
Se
  condicao:
    id:positivo
  entao:
    := y
      (+)
        id:y
        int:1
  senao:
    := y
      (-)
        id:y
        int:1
Enquanto
  condicao:
    (>)
      id:y
      int:0
  corpo:
    := y
      (-)
        id:y
        int:1
PS D:\-UNIVASF-Compilador-Pascal-TAM>
```

Note-se, na atribuição a y, como a precedência ficou registrada na estrutura: o nó (+) tem como filho esquerdo o nó (div) — ou seja, $x \text{ div } 2 + 3$ foi corretamente interpretado como $(x \text{ div } 2) + 3$.

Para o programa exemplos/procedimento-funcao.txt (Seção 1), com procedimento e função, a árvore impressa é:

Figura 6 – AST impressa no terminal (--ast)



```
PS D:\-UNIVASF-Compilador-Pascal-TAM> java -cp bin Compiler exemplos\procedimento-funcao.txt --ast
Programa: Subprogramas
Variaveis:
  var x: integer
  var ok: boolean
Subprogramas:
  procedure incrementar
    Bloco
      := x
      (+)
        id:x
        int:1
  function positivo: boolean
    Bloco
      := positivo
      (>)
        id:x
        int:0
Bloco
  := x
  int:1
  call incrementar
  := ok
  id:positivo
```

8. ANÁLISE DE CONTEXTO

A análise de contexto é realizada pela classe Checker, um `Visitor<TabelaSimbolos.Tipo>` que percorre a AST validando as dependências de contexto descritas na Seção 1.3. A fase engloba as duas subfases clássicas, identificação (associar cada uso de identificador à sua declaração) e verificação de tipos, realizadas em um único percurso da árvore. O método de entrada `verificar(programa)` devolve a tabela de símbolos preenchida, que alimenta a geração de código. Violações interrompem a análise com `ErroContexto`, contendo linha, coluna e mensagem específica.

8.1. Fase de identificação: estruturas de dados e algoritmos

A identificação apoia-se na TabelaSimbolos (Seção 6). Ao visitar as declarações, o Checker insere as entradas: visitarDeclaracaoVar registra cada nome da lista com a sua categoria (VARIABEL), o tipo declarado e o deslocamento sequencial; ao visitar um Bloco, os subprogramas são registrados (com categoria PROCEDIMENTO ou FUNCAO, tipo de retorno quando função e um rótulo único gerado pelo GeradorRotulos) antes de os seus corpos serem visitados — o que permite que o corpo de um subprograma chame outro declarado no mesmo bloco. A inserção rejeita duplicatas: declarar um identificador já existente lança “identificador 'x' ja foi declarado”.

Nos usos, a identificação é a consulta à tabela: visitarIdentificador (uso em expressão) chama buscarVariavelOuFuncao; visitarAtribuicao busca o alvo e valida a categoria (procedimento nunca recebe atribuição; função só dentro do próprio corpo — controle feito com o campo auxiliar funcaoAtual, que guarda o nome da função cujo corpo está sendo visitado); visitarChamadaProcedimento chama buscarProcedimento. Identificador ausente da tabela produz “identificador 'x' nao foi declarado”. Como o escopo da linguagem é único e global (decisão de projeto documentada na Seção 1.3.1), uma única tabela é suficiente, sem pilha de escopos.

Exemplo ilustrativo da identificação: no programa da Seção 7.2, ao processar o comando 'call incrementar', o Checker consulta a tabela, encontra a entrada (incrementar, PROCEDIMENTO, rótulo PROC_incrementar_0) e aceita a chamada; se o programador escrevesse 'incrementa' (não declarado), a análise seria interrompida com a mensagem de identificador não declarado, com a linha e a coluna do uso.

8.2. Fase de verificação de tipos: estruturas de dados e algoritmos

A verificação de tipos usa o próprio mecanismo de retorno do Visitor: cada método que visita uma expressão retorna o tipo da expressão (TabelaSimbolos.Tipo.INTEIRO ou BOOLEANO), calculado de baixo para cima (análise sintética). As folhas são imediatas — literal inteiro é INTEIRO; true/false é BOOLEANO; identificador retorna o tipo registrado na tabela; literal de texto é rejeitado. Nos nós internos, visitarBinario obtém os tipos dos dois filhos e aplica a

regra do operador (Tabela 2), com o método auxiliar `exigir(obtido, esperado, operador, lado)` padronizando as mensagens de erro; `visitarUnario` procede da mesma forma para `not` e `'-'` unário.

Nos comandos, `visitarAtribuicao` compara o tipo da expressão com o tipo declarado do alvo; `visitarSe` e `visitarEnquanto` exigem condição booleana, reportando o erro na linha do próprio `if/while` (token guardado no nó exatamente para isso). Exemplo ilustrativo: em `'x := true'` com `x` inteiro, o filho direito retorna `BOOLEANO`, o tipo declarado é `INTEIRO`, e a comparação falha com a mensagem de tipos incompatíveis; em `'if x then ...'` com `x` inteiro, a condição retorna `INTEIRO` e a exigência de booleano falha.

8.3. Exemplos de entradas e saídas

8.3.1 Aceitação de cadeias bem formadas

Os programas `exemplos/valido1.txt`, `exemplos/valido2.txt` e `exemplos/procedimento-funcao.txt` passam integralmente pela análise de contexto: a execução com `--code` prossegue até a geração de código e termina com código de saída 0 e a mensagem “Codigo TAM gerado em: ...” (Seção 11.3), o que comprova a aceitação.

8.3.2 Rejeição de cadeias mal formadas

Todas as saídas abaixo são de execuções reais do compilador. Para `exemplos/erro-semantic.txt`:

```
program ErroSemantico;
var
  x: integer;
begin
  x := true
end.
```

a execução com `--code` produz:

```
Erro de contexto na linha 5, coluna 3: tipo incompativel na atribuicao de 'x': variavel e inteiro,
expressao e booleano
```

Demais violações testadas, com as respectivas saídas reais:

Tabela 9 – Violações de contexto testadas e mensagens emitidas

Programa de teste (trecho)	Saída real do compilador
y := 1 (y não declarado)	Erro de contexto na linha 3, coluna 3: identificador 'y' nao foi declarado
var x: integer; x: boolean;	Erro de contexto na linha 3, coluna 1: identificador 'x' ja foi declarado
if x then ... (x inteiro)	Erro de contexto na linha 4, coluna 3: condicao do 'if' deve ser booleana, mas e inteiro
x := 'ola'	Erro de contexto na linha 4, coluna 8: literal de texto nao e suportado: a linguagem aceita apenas 'integer' e 'boolean'
x := true and 1	Erro de contexto na linha 4, coluna 13: operando direita de 'and' deve ser booleano, mas e inteiro

Figura 7 – Mensagem de erro de contexto no terminal

```
PS C:\Users\Andrea\Documents\Github\UNIVASF-Compilador-Pascal-TAM> java -cp bin Compiler exemplos\erro-semantic.txt
--code
Erro de contexto na linha 5, coluna 3: tipo incompativel na atribuicao de 'x': variavel e inteiro, expressao e boolea
no
PS C:\Users\Andrea\Documents\Github\UNIVASF-Compilador-Pascal-TAM> |
```

9. AMBIENTE DE EXECUÇÃO

9.1. Tipo de máquina usada

A máquina-objeto adotada é a TAM (Triangle Abstract Machine), definida no livro-texto (WATT; BROWN, 2000, Apêndice C), no modelo de máquina de pilha. Na TAM, os dados residem em uma pilha: as instruções não operam sobre registradores de propósito geral, mas empilham e desempilham valores. A máquina organiza a memória em regiões apontadas por registradores dedicados, entre eles CB (Code Base, início do código), SB (Stack Base, início da pilha, onde vivem as variáveis globais), ST (Stack Top, topo corrente) e PB (Primitive Base, endereço-base do conjunto de rotinas primitivas que implementam os operadores aritméticos, lógicos e relacionais). O compilador gera um arquivo de texto com instruções TAM simbólicas, uma por linha, em que as variáveis são referenciadas pelos seus nomes, conforme requisito do enunciado.

9.2. Avaliação de expressões

As expressões são avaliadas na pilha, em notação pós-fixa: o código gerado primeiro empilha os operandos (da esquerda para a direita) e em seguida invoca a rotina primitiva do operador, que desempilha os operandos e empilha o resultado. Por exemplo, a expressão $x \div 2 + 3$ gera `LOAD x; LOADL 2; CALL(SB) 10[PB]` (div, deixando $x \div 2$ no topo); `LOADL 3; CALL(SB) 7[PB]` (add, deixando o resultado final no topo). Essa disciplina garante que, ao final da avaliação de qualquer expressão, exatamente o seu valor esteja no topo da pilha, invariante usada pelos comandos que consomem expressões (atribuição, condições de `if` e `while`).

9.3. Representação de dados

Ambos os tipos da linguagem ocupam uma palavra da máquina. Os inteiros são representados diretamente pelo seu valor. Os booleanos seguem a convenção da TAM: `true` é representado por 1 e `false` por 0, convenção aplicada pelo gerador ao traduzir literais booleanos (`LOADL 1 / LOADL 0`) e assumida pela instrução de desvio condicional `JUMPIF(0)`, que desvia quando o topo da pilha é 0 (condição falsa). Literais de texto não possuem representação em tempo de execução, pois são rejeitados na análise de contexto.

9.4. Formas de alocação de memória empregadas

A alocação é estática/global: todas as variáveis do programa são alocadas de uma única vez, no início da execução do código principal, pela instrução `PUSH x, y, ...` — que reserva uma palavra por variável, na ordem de declaração (ordem preservada pelo `LinkedHashMap` da tabela de símbolos). O deslocamento de cada variável em relação a SB fica registrado na tabela de símbolos e documenta a posição de cada nome na pilha, embora as instruções geradas referenciem as variáveis pelo nome (`LOAD x, STORE y`), conforme o requisito. Não há alocação dinâmica (heap) nem quadros de ativação (frames) com variáveis locais: como os subprogramas não possuem parâmetros nem declarações locais próprias no uso que a linguagem faz deles, os seus corpos operam sobre as variáveis globais.

9.5. Passagem de parâmetros e retorno de valor de função

Por decisão de projeto, procedimentos e funções não recebem parâmetros; a comunicação entre chamador e subprograma é feita pelas variáveis globais. A chamada de procedimento gera CALL rótulo e o corpo termina em RETURN, que devolve o controle à instrução seguinte à chamada.

O retorno de valor de função segue o estilo Pascal: dentro do corpo, o valor é definido por atribuição ao nome da função (positivo := $x > 0$), armazenado com STORE positivo como se o nome fosse uma variável. Ao final do corpo da função o gerador emite LOAD nome; RETURN — ou seja, o valor retornado é deixado no topo da pilha ao retornar. No ponto de uso (a função aparece como fator numa expressão), o gerador emite CALL FUNC_rotulo e o valor fica disponível no topo para a continuação da avaliação — por exemplo, ok := positivo gera CALL FUNC_positivo_1; STORE ok.

10. LINGUAGEM-OBJETO

A linguagem-objeto é uma linguagem textual de instruções da máquina TAM, gravada em arquivo .txt com uma instrução (ou rótulo) por linha. As variáveis são referenciadas pelos seus nomes e os destinos de desvio, por rótulos simbólicos, o que torna o código legível e independente de endereços numéricos.

10.1. Relação de instruções

Tabela 10 – Instruções da linguagem-objeto

Instrução	Sintaxe	Semântica
PUSH	PUSH nome1, nome2, ...	Reserva na pilha uma palavra para cada variável listada, na ordem de declaração
LOADL	LOADL v	Empilha o literal inteiro v (booleanos: 1 = true, 0 = false)
LOAD	LOAD nome	Empilha o valor corrente da variável nome
STORE	STORE nome	Desempilha o topo e armazena na variável nome
CALL(SB) n[PB]	CALL(SB) n[PB]	Invoca a rotina primitiva de número n (Tabela 11); desempilha os operandos e empilha o resultado

CALL	CALL rótulo	Chama o subprograma cujo corpo inicia no rótulo, guardando o endereço de retorno
JUMP	JUMP rótulo	Desvio incondicional para o rótulo
JUMPIF(0)	JUMPIF(0) rótulo	Desempilha o topo; desvia para o rótulo se o valor for 0 (falso)
RETURN	RETURN	Retorna do subprograma para a instrução seguinte à chamada
HALT	HALT	Encerra a execução do programa
Rótulo	nome:	Marca simbólica de posição no código; não é executada

10.2. Sintaxe

Um programa-objeto é uma sequência de linhas; cada linha contém um rótulo (identificador seguido de dois-pontos) ou uma instrução (mnemônico seguido, quando aplicável, dos operandos separados por vírgula). Os operandos podem ser literais inteiros (LOADL 10), nomes de variáveis (LOAD x, STORE y), referências a primitivas na forma n[PB] (CALL(SB) 7[PB]) ou rótulos (JUMP MAIN_0, CALL PROC_incrementar_0). Os rótulos gerados seguem o padrão PREFIXO_n, com prefixos MAIN, ELSE, FIM_IF, WHILE, FIM_WHILE, PROC_nome e FUNC_nome, e numeração única fornecida pelo gerador de rótulos.

10.3. Semântica

A execução é sequencial a partir da primeira linha, com a semântica de pilha descrita na Seção 9: LOADL/LOAD empilham, STORE desempilha, CALL(SB) n[PB] aplica o operador primitivo aos operandos no topo da pilha e JUMPIF(0) consome o topo como condição. Quando o programa possui subprogramas, a primeira instrução é JUMP MAIN_n, saltando sobre os corpos dos subprogramas para iniciar a execução no código principal; os corpos são executados apenas via CALL. A execução termina em HALT. As rotinas primitivas usadas pelo gerador são as do Apêndice C do livro-texto:

Tabela 11 – Rotinas primitivas da TAM utilizadas

Endereço	Primitiva	Operação
1[PB]	not	Negação lógica do topo
2[PB]	and	Conjunção lógica dos dois valores do topo
3[PB]	or	Disjunção lógica dos dois valores do topo
6[PB]	neg	Negação aritmética do topo (menos unário)
7[PB]	add	Soma
8[PB]	sub	Subtração
9[PB]	mul	Multiplicação
10[PB]	div	Divisão inteira
11[PB]	mod	Resto da divisão inteira
12[PB]	lt	Menor que (<)
13[PB]	le	Menor ou igual (<=)
14[PB]	ge	Maior ou igual (>=)
15[PB]	gt	Maior que (>)
16[PB]	eq	Igual (=)
17[PB]	ne	Diferente (<>)

11. GERAÇÃO DE CÓDIGO

11.1 Funções de código implementadas

A geração segue o esquema de funções de código do livro-texto, em que cada categoria de construção da linguagem possui uma função geradora com um padrão de código definido. No projeto, essas funções são os métodos visitarX da classe Coder (o Visitor faz o papel do despacho entre as funções). Os padrões implementados são:

Tabela 12 – Padrões de código por construção da linguagem

Construção	Padrão de código gerado
Programa (com subprogramas)	JUMP MAIN_n · corpos dos subprogramas · MAIN_n: · PUSH vars · código do corpo · HALT
Programa (sem subprogramas)	PUSH vars · código do corpo · HALT
Atribuição $v := E$	código de E · STORE v
Chamada de procedimento p	CALL PROC_p_n

if E then C1 (sem else)	código de E · JUMPIF(0) FIM_IF_n · código de C1 · FIM_IF_n:
if E then C1 else C2	código de E · JUMPIF(0) ELSE_n · código de C1 · JUMP FIM_IF_m · ELSE_n: · código de C2 · FIM_IF_m:
while E do C	WHILE_n: · código de E · JUMPIF(0) FIM_WHILE_m · código de C · JUMP WHILE_n · FIM_WHILE_m:
Comando composto	concatenação do código dos comandos, em ordem
Comando vazio	nenhuma instrução
Procedimento p	PROC_p_n: · código do corpo · RETURN
Função f	FUNC_f_n: · código do corpo · LOAD f · RETURN
Expressão binária E1 op E2	código de E1 · código de E2 · CALL(SB) prim(op)[PB]
Expressão unária op E	código de E · CALL(SB) prim(op)[PB]
Identificador (variável)	LOAD nome
Identificador (função em expressão)	CALL FUNC_f_n
Literal inteiro k	LOADL k
Literais true / false	LOADL 1 / LOADL 0

11.2 Estruturas de dados e algoritmos utilizados

O Coder recebe no construtor a tabela de símbolos preenchida pelo Checker e acumula as instruções em um `ArrayList<String>` (método `emitir`; `emitirRotulo` acrescenta linhas de rótulo). O gerador de rótulos exigido pelo enunciado é a classe `GeradorRotulos`: um contador único que produz rótulos distintos por prefixo (Seção 10.2). Como os desvios referenciam rótulos simbólicos — e não endereços numéricos —, não é necessária retrocorreção (`backpatching`): o rótulo é criado antes da emissão do desvio e a sua linha é emitida na posição correta do fluxo. O método `emitirAlocacaoVariaveis` percorre a tabela de símbolos (em ordem de declaração) e monta a instrução `PUSH` com os nomes de todas as variáveis.

O algoritmo é um único percurso da AST: `visitarPrograma` decide se emite o salto inicial `JUMP MAIN_n` (apenas quando há subprogramas), emite os corpos dos subprogramas, o rótulo principal, a alocação, o código do corpo e `HALT`. Os métodos de expressão implementam a avaliação pós-fixa (Seção 9.2) e consultam a tabela para distinguir variável (`LOAD nome`) de função (`CALL rotulo`) no uso de identificadores. Ao final, a classe `Compiler` grava a lista de instruções no arquivo de saída `<nome-do-fonte>-tam.txt`, na mesma pasta do arquivo-fonte, e informa no console o caminho gerado e o total de instruções.

11.3 Exemplos de tradução

Executando `java -cp bin Compiler exemplos\valido2.txt --code`, o compilador imprime “Codigo TAM gerado em: exemplos\valido2-tam.txt / Total de instrucoes: 38” e grava o arquivo abaixo (conteúdo real, na íntegra). À esquerda, o programa-fonte; à direita, o código-objeto:

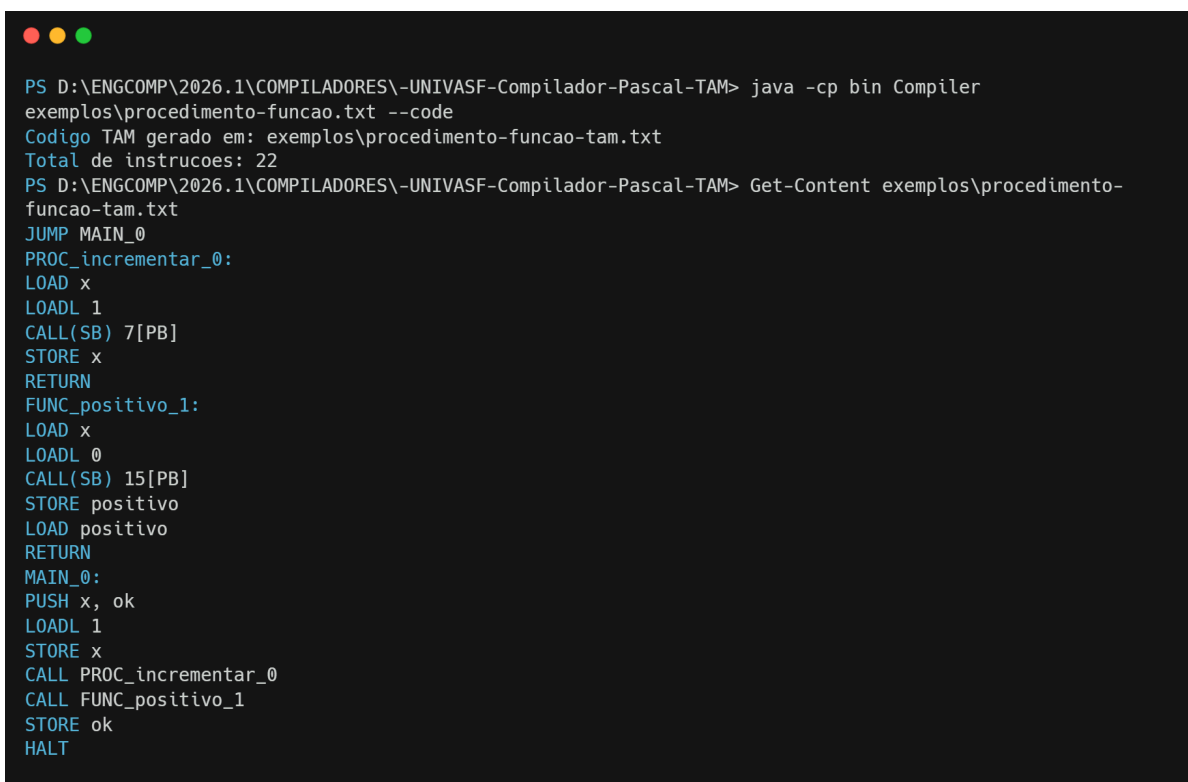
Tabela 13 – Tradução completa de exemplos/valido2.txt

Programa-fonte	Código TAM gerado (valido2-tam.txt)
<pre> program Valido; var x, y: integer; positivo: boolean; begin x := 10; y := x div 2 + 3; positivo := y >= 0; if positivo then y := y + 1 else y := y - 1; while y > 0 do y := y - 1 end. </pre>	<pre> PUSH x, y, positivo LOADL 10 STORE x LOAD x LOADL 2 CALL(SB) 10[PB] LOADL 3 CALL(SB) 7[PB] STORE y LOAD y LOADL 0 CALL(SB) 14[PB] STORE positivo LOAD positivo JUMPIF(0) ELSE_1 LOAD y LOADL 1 CALL(SB) 7[PB] STORE y JUMP FIM_IF_2 ELSE_1: LOAD y LOADL 1 CALL(SB) 8[PB] STORE y FIM_IF_2: WHILE_3: LOAD y LOADL 0 CALL(SB) 15[PB] JUMPIF(0) FIM_WHILE_4 LOAD y LOADL 1 CALL(SB) 8[PB] STORE y JUMP WHILE_3 FIM_WHILE_4: HALT </pre>

Observam-se no exemplo os padrões da Tabela 12: a avaliação pós-fixa de $x \text{ div } 2 + 3$ (div antes de add, refletindo a precedência); o if/else com JUMPIF(0) para o rótulo ELSE_1 e salto JUMP FIM_IF_2 ao final do ramo verdadeiro; e o while com rótulo de teste WHILE_3 no início e JUMPIF(0) FIM_WHILE_4 na saída.

Para o programa com subprogramas (java -cp bin Compiler exemplos\procedimento-funcao.txt --code, 22 instruções), o arquivo gerado é *Aqui se veem o salto inicial sobre os corpos dos subprogramas (JUMP MAIN_0), o padrão de procedimento (rótulo, corpo, RETURN), o padrão de função (corpo, LOAD positivo para deixar o valor de retorno no topo, RETURN) e o uso da função em expressão: ok := positivo vira CALL FUNC_positivo_1 seguido de STORE ok.*

Figura 8 – Arquivo de código-objeto gerado



```
PS D:\ENGCAMP\2026.1\COMPILADORES\UNIVASF-Compilador-Pascal-TAM> java -cp bin Compiler
exemplos\procedimento-funcao.txt --code
Codigo TAM gerado em: exemplos\procedimento-funcao-tam.txt
Total de instrucoes: 22
PS D:\ENGCAMP\2026.1\COMPILADORES\UNIVASF-Compilador-Pascal-TAM> Get-Content exemplos\procedimento-
funcao-tam.txt
JUMP MAIN_0
PROC_incrementar_0:
LOAD x
LOADL 1
CALL(SB) 7[PB]
STORE x
RETURN
FUNC_positivo_1:
LOAD x
LOADL 0
CALL(SB) 15[PB]
STORE positivo
LOAD positivo
RETURN
MAIN_0:
PUSH x, ok
LOADL 1
STORE x
CALL PROC_incrementar_0
CALL FUNC_positivo_1
STORE ok
HALT
```

12. MANUAL DE COMPILAÇÃO

Esta seção descreve como obter a versão executável do compilador a partir dos programas-fonte fornecidos, tanto em Windows quanto em Linux.

12.1 Requisitos de ambiente

Tabela 14 – Recursos necessários para compilar o projeto

Recurso	Versão / observação
Sistema operacional	Windows 10/11 ou Linux (testado em Windows 11 e Ubuntu 22.04)
JDK (Java Development Kit)	JDK 17 LTS ou superior (o código usa recursos até o Java 11, como Files.readString e Path.of; recomenda-se um LTS recente — o projeto foi desenvolvido com JDK atual no Windows e validado também com JDK 17 no Linux)
PowerShell	5.1 ou superior — necessário apenas para o script de testes scripts/run-tests.ps1 (Windows)
Editor de texto	Qualquer um (VS Code, Notepad++, nano etc.), apenas para editar fontes e exemplos

Não há dependências externas: o projeto usa exclusivamente a biblioteca-padrão do Java, não requer Maven, Gradle ou pacotes adicionais.

12.2 Estrutura de pastas do projeto

-UNIVASF-Compilador-Pascal-TAM/

— src/	código-fonte Java do compilador
— exemplos/	programas-fonte de teste e saídas -tam.txt
— scripts/	run-tests.ps1 (suíte de testes automatizada)
— docs/	documentação interna das etapas
— bin/	classes compiladas (criada pela compilação)

12.3 Compilação no Windows

Abra o PowerShell (ou o Prompt de Comando) na pasta raiz do projeto e execute:

```
javac -encoding UTF-8 -d bin src\*.java
```

O comando invoca o compilador Java (javac) informando que os fontes usam a codificação UTF-8 (-encoding UTF-8, necessário por causa dos comentários acentuados no código) e que as classes compiladas devem ser gravadas na pasta bin (-d bin), que é criada automaticamente. A compilação bem-sucedida não imprime nada; os arquivos .class ficam disponíveis em bin/.

12.4 Compilação no Linux

Instale o JDK (por exemplo, em distribuições baseadas em Debian/Ubuntu: `sudo apt install openjdk-17-jdk`) e, na pasta raiz do projeto, execute:


```
javac -encoding UTF-8 -d bin src/*.java
```

A única diferença em relação ao Windows é o separador de caminho (barra em vez de contrabarra). O resultado é idêntico: pasta bin/ com as classes compiladas.

12.5 Verificação da compilação

Para confirmar que a versão executável foi obtida corretamente, execute o compilador sobre um exemplo fornecido:

```
java -cp bin Compiler exemplos\valido1.txt    (Windows)
java -cp bin Compiler exemplos/valido1.txt    (Linux)
```

A saída esperada é “Análise léxica concluída sem erros.”, seguida das dicas de uso das demais opções. Alternativamente, no Windows, execute a suíte completa de testes (Seção 12.6).

12.6 Script de testes (run-tests.ps1)

O projeto inclui o script PowerShell scripts/run-tests.ps1, que compila o projeto e executa a bateria de testes automatizados: análise léxica de programa válido, erro léxico por caractere inválido, erro léxico por comentário aberto, AST de programa válido, erro sintático, geração de código de valido2 (verificando a existência do arquivo -tam.txt e a presença de PUSH, HALT, rótulos e variáveis por nome), tradução de procedimento e função e erro de contexto. Para executá-lo:

```
powershell -ExecutionPolicy Bypass -File scripts/run-tests.ps1
```

O parâmetro -ExecutionPolicy Bypass permite executar o script sem alterar a política de execução global do sistema. Ao final, o script reporta cada teste como aprovado ou reprovado.

Figura 9 – Execução da suíte de testes automatizada

```
PS C:\Users\Andrea\Documents\Github\UNIVASF-Compilador-Pascal-TAM> powershell -ExecutionPolicy Bypass -File scripts\run-tests.ps1
[OK] valido1
[OK] erro lexico caractere
[OK] erro comentario aberto
Todos os testes base passaram.
[OK] ast valido1 imprime programa
[OK] ast valido1 imprime variaveis
[OK] ast valido1 imprime bloco
[OK] ast valido1 imprime Se
[OK] ast valido1 imprime Enquanto
[OK] ast erro sintatico detectado
Todos os testes da etapa B passaram.
[OK] code valido2 gera arquivo tam
[OK] code procedimento funcao
[OK] code valido2 arquivo existe
[OK] code valido2 contem PUSH
[OK] code valido2 contem HALT
[OK] code valido2 contem rotulo
[OK] code valido2 usa nomes
[OK] code procedimento contem rotulo
[OK] code funcao contem rotulo
[OK] code chama procedimento
[OK] code erro semantico detectado
[OK] code valido1 falha no checker
Todos os testes das etapas C e D passaram.
```

13. MANUAL DE INSTALAÇÃO

O compilador não requer instalador, registro no sistema ou serviços: a instalação consiste em ter um JDK funcional e uma cópia da pasta do projeto. Os passos completos são descritos a seguir.

13.1 Passo 1 — Instalar o JDK

- **Windows:** baixe um JDK 17 (ou **superior**) — por exemplo, Eclipse Temurin (adoptium.net) ou Oracle JDK — e execute o instalador. Marque, quando disponível, as opções de configurar JAVA_HOME e adicionar o Java ao PATH. Caso o instalador não o faça, configure manualmente: em Configurações → Sistema → Sobre → Configurações avançadas do sistema → Variáveis de Ambiente, crie a variável JAVA_HOME apontando para a pasta de instalação do JDK (por exemplo, C:\Program Files\Eclipse Adoptium\jdk-17...) e acrescente %JAVA_HOME%\bin à variável Path.
- **Linux (Debian/Ubuntu):** execute `sudo apt update && sudo apt install openjdk-17-jdk`. O pacote já configura os executáveis no PATH. Se desejar definir JAVA_HOME, acrescente ao `~/.bashrc`: `export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64` e `export PATH=$JAVA_HOME/bin:$PATH`.

Verifique a instalação em um terminal novo:

```
java -version  
javac -version
```

Ambos os comandos devem responder com a versão instalada (por exemplo, “openjdk version “17.0.19”).

13.2 Editor de Código:

- Utilize o Visual Studio Code como editor de código, que está disponível gratuitamente para download: [Visual Studio Code Download](#).

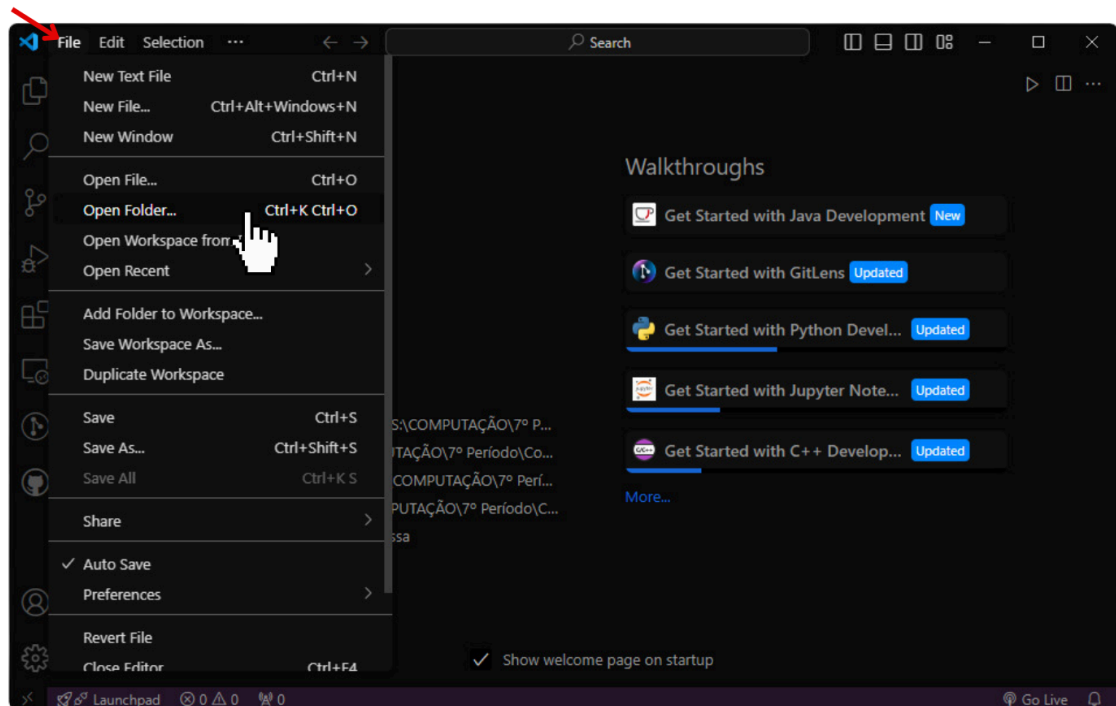
Configurações do Visual Studio Code:

- Siga as recomendações da documentação do Visual Studio Code para compilar e executar um código Java. A documentação fornece uma descrição completa e os passos necessários para configurar o ambiente, adicionar as extensões e compilar o código. Todos os downloads necessários estão disponíveis no site da documentação: [Java no Visual Studio Code](#).
- Após realizar todos os passos acima, o ambiente para compilar e executar o compilador estará pronto. Baixe o código-fonte do compilador de no repositório [GitHub](#), ou extraia a pasta do pendrive no diretório desejado e siga as instruções abaixo para finalizar a configuração e executar o compilador:

1. Configurando o Ambiente no Visual Studio Code:

- Abra o Visual Studio Code e clique em "Files" (ou "Arquivos") e selecione "Open Folder" (ou "Abrir Pasta") conforme mostrado na figura 10.

Figura 10 - Passo a passo da abertura do projeto no Visual Studio Code



2. Abrir a Pasta do Projeto:

- Navegue até o local da pasta do projeto, selecione-a e abra.

3. Executar o Compilador:

- Pronto, os arquivos do projeto estarão disponíveis no editor.
- Abra a pasta src e abra o arquivo Compiler.java, aguarde o Visual Studio Code Ativar as funcionalidades para o Java.

13.3 Passo 3 — Compilar e validar

- Siga a Seção 12 para gerar a pasta bin/ e valide com um exemplo fornecido. A partir daí o compilador está instalado e operacional. Para desinstalar, basta remover a pasta do projeto.

14. MANUAL DE UTILIZAÇÃO

14.1. Requisitos do sistema

Para utilizar o compilador são necessários: Windows 10/11 ou Linux; JRE/JDK 17 ou superior (para executar o JDK é necessário apenas para compilar o projeto); e a pasta do projeto com bin/ gerada conforme a Seção 12. Não há bibliotecas externas. O terminal pode ser o PowerShell, o Prompt de Comando (Windows) ou qualquer shell (Linux).

14.2. Criando e localizando um arquivo-fonte

Um arquivo-fonte é um arquivo de texto simples com um programa na linguagem descrita na Seção 1. Crie-o com qualquer editor de texto e salve-o, de preferência, na pasta exemplos/ do projeto — mas qualquer pasta serve, desde que o caminho seja informado na linha de comando. Por convenção do projeto utiliza-se a extensão .txt (todos os exemplos fornecidos a usam); entretanto, o compilador não exige extensão específica: qualquer nome de arquivo legível é aceito. Recomenda-se salvar em codificação UTF-8. Exemplo de criação:

```
notepad exemplos\meuteste.txt      (Windows)
nano exemplos/meuteste.txt         (Linux)
```

Conteúdo mínimo de um programa válido:

```
program Minimo;
begin
end.
```

14.3. Formas de uso e opções de compilação

A forma geral de invocação, a partir da pasta raiz do projeto, é:

```
java -cp bin Compiler <arquivo-fonte> [opções]
```

As opções controlam até que fase o processamento avança, atendendo aos níveis de uso exigidos:

Tabela 15 – Opções de linha de comando e fases executadas

Uso desejado	Comando	Comportamento
Análise léxica apenas	java -cp bin Compiler exemplos\valido2.txt	Executa somente o léxico; imprime “Análise léxica

		concluída sem erros.” ou o erro léxico encontrado
Léxico com listagem de tokens	<code>java -cp bin Compiler exemplos\valido2.txt --tokens</code>	Executa o léxico e imprime a sequência completa de tokens (tipo, lexema, linha, coluna)
Até análise sintática / impressão da árvore	<code>java -cp bin Compiler exemplos\valido2.txt --ast</code>	Executa léxico e sintático e imprime a AST indentada; erros interrompem com mensagem
Até análise de contexto	<code>java -cp bin Compiler exemplos\erro-semantic.txt --code</code>	A verificação de contexto é executada como primeira etapa de <code>--code</code> ; se houver violação, o processamento para na mensagem de erro de contexto e nenhum código é gerado
Até geração de código	<code>java -cp bin Compiler exemplos\valido2.txt --code</code>	Executa todas as fases e grava o arquivo <nome>-tam.txt na pasta do arquivo-fonte
Ajuda	<code>java -cp bin Compiler --help</code>	Mostra o resumo de uso e as opções

Observação sobre a análise sintática apenas: a opção `--ast` atende simultaneamente aos níveis “até análise sintática” e “até impressão da árvore”, pois a árvore é construída durante a própria análise e a impressão é a evidência da aceitação.

Observação sobre a análise de contexto: o compilador não possui uma opção isolada que pare após o contexto sem gerar código; a verificação de contexto está embutida em `--code` e, em caso de programa válido, o código é gerado em seguida. Para usar o compilador como verificador, basta ignorar o arquivo gerado.

As opções podem ser combinadas (por exemplo, `--tokens --ast` imprime os tokens e depois a árvore). No Linux, use barras nos caminhos (exemplos/valido2.txt).

14.4. Onde as saídas são apresentadas

Todas as saídas informativas (lista de tokens, AST, mensagens de progresso e a confirmação “Codigo TAM gerado em: ...”) são apresentadas na tela, no fluxo de

saída padrão (stdout). Todas as mensagens de erro são apresentadas na tela, no fluxo de erro padrão (stderr). O único artefato gravado em disco é o código-objeto: um arquivo de texto chamado <nome-do-fonte>-tam.txt, criado na mesma pasta do arquivo-fonte (por exemplo, exemplos/valido2.txt gera exemplos/valido2-tam.txt). O processo termina com código de saída 0 (sucesso), 1 (erro léxico, sintático ou de contexto) ou 2 (erro de uso da linha de comando ou de leitura/escrita de arquivo), o que permite integrar o compilador a scripts.

14.5. Mensagens de erro: causas e soluções

As mensagens seguem o formato “Erro <fase> na linha L, coluna C: <descrição>”, sempre com a posição exata do problema no arquivo-fonte. As tabelas a seguir descrevem todas as mensagens emitidas pelo compilador, com causa e solução.

Tabela 16 – Erros léxicos

Mensagem	Causa	Solução
caractere inesperado 'X'	O caractere X não pertence ao alfabeto da linguagem (ex.: @, #, \$)	Remover ou substituir o caractere inválido
comentário iniciado com '{' não foi fechado	Um comentário { ... foi aberto e o arquivo terminou sem o }	Fechar o comentário com }
comentário iniciado com '(' não foi fechado	Um comentário (* ... foi aberto e o arquivo terminou sem *)	Fechar o comentário com *)
numero inteiro não pode ser seguido diretamente por letra ou '_'	Literal como 2abc (número colado em letra)	Separar o número do identificador com espaço ou operador
literal de string não foi fechado antes do fim da linha	Aspa simples aberta e quebra de linha antes do fechamento	Fechar a aspa na mesma linha
literal de string não foi fechado	Aspa simples aberta e arquivo terminou	Fechar a aspa simples

Tabela 17 – Erros sintáticos

Mensagem	Causa	Solução
esperava 'X', encontrado 'Y'	A estrutura do programa viola a gramática: o token X era obrigatório naquele ponto (ex.: ';' faltando após o cabeçalho)	Inserir/corrigir o token indicado na posição apontada
esperava pelo menos uma declaração de variável após 'var'	Seção var presente, porém vazia	Declarar ao menos uma variável ou remover o var

tipo esperado ('integer' ou 'boolean'), encontrado 'X'	Denotador de tipo inválido na declaração (ex.: real, string)	Usar integer ou boolean
expressão esperada, encontrado 'X'	Faltou uma expressão onde a gramática exige (ex.: x := ;)	Completar a expressão

Tabela 18 – Erros de contexto

Mensagem	Causa	Solução
identificador 'x' não foi declarado	Uso de variável/subprograma sem declaração	Declarar o identificador na seção var ou como subprograma
identificador 'x' já foi declarado	Declaração duplicada (variável ou subprograma)	Renomear ou remover a segunda declaração
'x' e procedimento e não recebe atribuição	Nome de procedimento usado como alvo de :=	Usar uma variável como alvo; procedimentos só são chamados
'x' e função e so pode receber atribuição dentro do proprio corpo	Atribuição ao nome de uma função fora do corpo dela	Mover a atribuição para dentro do corpo da função (definição do retorno)
tipo incompatível na atribuição de 'x': variavel e T1, expressao e T2	O tipo da expressão difere do tipo declarado da variável	Ajustar a expressão ou o tipo declarado
'x' nao pode ser usado como expressao	Procedimento usado como fator em expressão	Usar uma função ou variável
'x' nao e procedimento	Variável ou função usada como comando de chamada	Chamar apenas procedimentos; funções são usadas em expressões
condicao do 'if' deve ser booleana, mas e inteiro	Expressão não booleana como condição de if	Usar expressão booleana (ex.: x > 0)
condicao do 'while' deve ser booleana, mas e inteiro	Expressão não booleana como condição de while	Usar expressão booleana
operando esquerda/direita de 'op' deve ser T, mas e U	Operando com tipo incompatível com o operador (Tabela 2)	Ajustar os operandos ao tipo exigido pelo operador
operandos de '=' (ou '<>') devem ter o mesmo tipo	Comparação de igualdade entre inteiro e booleano	Comparar valores do mesmo tipo
literal de texto nao e suportado: a linguagem aceita apenas 'integer' e 'boolean'	Uso de literal 'texto' em expressão	Remover o literal de texto; a linguagem não possui tipo string

Tabela 19 – Erros de linha de comando e de E/S

Mensagem	Causa	Solução
Arquivo-fonte nao informado.	Nenhum caminho de arquivo foi passado	Informar o caminho do arquivo-fonte

Opcao desconhecida: --x	Opção inexistente na linha de comando	Usar --tokens, --ast, --code ou --help
Mais de um arquivo-fonte informado.	Dois ou mais caminhos foram passados	Compilar um arquivo por vez
Nao foi possivel ler o arquivo: <caminho>	Arquivo inexistente ou sem permissão de leitura	Conferir o caminho e as permissões
Nao foi possivel escrever: <caminho>	Pasta de saída sem permissão de escrita	Executar em pasta com permissão de escrita

14.6. Exemplo de sessão completa de uso

Figura 11, 12 e 13 – Sessão completa de uso do compilador

```
PS D:\-UNIVASF-Compilador-Pascal-TAM> javac -encoding UTF-8 -d bin src\*.java
PS D:\-UNIVASF-Compilador-Pascal-TAM> java -cp bin Compiler exemplos\valido2.txt --tokens
PROGRAMA      "program"      linha 1, coluna 1
IDENTIFICADOR "Valido"      linha 1, coluna 9
PONTO_E_VIRGULA ";"        linha 1, coluna 15
VAR           "var"        linha 2, coluna 1
IDENTIFICADOR "x"         linha 3, coluna 3
VIRGULA       ","         linha 3, coluna 4
IDENTIFICADOR "y"         linha 3, coluna 6
DOIS_PONTOS   ":"         linha 3, coluna 7
INTEIRO       "integer"   linha 3, coluna 9
PONTO_E_VIRGULA ";"        linha 3, coluna 16
IDENTIFICADOR "positivo"   linha 4, coluna 3
DOIS_PONTOS   ":"         linha 4, coluna 11
BOOLEANO      "boolean"   linha 4, coluna 13
PONTO_E_VIRGULA ";"        linha 4, coluna 20
INICIO        "begin"     linha 5, coluna 1
IDENTIFICADOR "x"         linha 6, coluna 3
ATRIBUICAO    ":@"        linha 6, coluna 5
LITERAL_INTEIRO "10"      linha 6, coluna 8
PONTO_E_VIRGULA ";"        linha 6, coluna 10
IDENTIFICADOR "y"         linha 7, coluna 3
ATRIBUICAO    ":@"        linha 7, coluna 5
IDENTIFICADOR "x"         linha 7, coluna 8
DIV           "div"       linha 7, coluna 10
LITERAL_INTEIRO "2"       linha 7, coluna 14
MAIS          "+"        linha 7, coluna 16
LITERAL_INTEIRO "3"       linha 7, coluna 18
PONTO_E_VIRGULA ";"        linha 7, coluna 19
IDENTIFICADOR "positivo"   linha 8, coluna 3
ATRIBUICAO    ":@"        linha 8, coluna 12
IDENTIFICADOR "y"         linha 8, coluna 15
MAIOR_IGUAL   ">="        linha 8, coluna 17
LITERAL_INTEIRO "0"       linha 8, coluna 20
PONTO_E_VIRGULA ";"        linha 8, coluna 21
SE            "if"        linha 9, coluna 3
IDENTIFICADOR "positivo"   linha 9, coluna 6
ENTAO         "then"      linha 9, coluna 15
IDENTIFICADOR "y"         linha 10, coluna 5
ATRIBUICAO    ":@"        linha 10, coluna 7
IDENTIFICADOR "y"         linha 10, coluna 10
MAIS          "+"        linha 10, coluna 12
LITERAL_INTEIRO "1"       linha 10, coluna 14
SENAO         "else"      linha 11, coluna 3
IDENTIFICADOR "y"         linha 12, coluna 5
ATRIBUICAO    ":@"        linha 12, coluna 7
IDENTIFICADOR "y"         linha 12, coluna 10
MENOS         "-"        linha 12, coluna 12
LITERAL_INTEIRO "1"       linha 12, coluna 14
PONTO_E_VIRGULA ";"        linha 12, coluna 15
ENQUANTO      "while"     linha 13, coluna 3
IDENTIFICADOR "y"         linha 13, coluna 9
MAIOR         ">"         linha 13, coluna 11
LITERAL_INTEIRO "0"       linha 13, coluna 13
FACA          "do"        linha 13, coluna 15
IDENTIFICADOR "y"         linha 14, coluna 5
ATRIBUICAO    ":@"        linha 14, coluna 7
IDENTIFICADOR "y"         linha 14, coluna 10
MENOS         "-"        linha 14, coluna 12
LITERAL_INTEIRO "1"       linha 14, coluna 14
FIM           "end"       linha 15, coluna 1
PONTO         "."         linha 15, coluna 4
FIM_ARQUIVO   ""         linha 16, coluna 1
```

```
PS D:\-UNIVASF-Compilador-Pascal-TAM> java -cp bin Compiler exemplos\valido2.txt --ast
Programa: Valido
Variaveis:
  var x, y: integer
  var positivo: boolean
Bloco
  := x
    int:10
  := y
    (+)
      (div)
        id:x
        int:2
      int:3
  := positivo
    (>=)
      id:y
      int:0
Se
  condicao:
    id:positivo
  entao:
    := y
      (+)
        id:y
        int:1
  senao:
    := y
      (-)
        id:y
        int:1
Enquanto
  condicao:
    (>)
      id:y
      int:0
  corpo:
    := y
      (-)
        id:y
        int:1
```

```
PS D:\-UNIVASF-Compilador-Pascal-TAM> java -cp bin Compiler exemplos\valido2.txt --code
Codigo TAM gerado em: exemplos\valido2-tam.txt
Total de instrucoes: 38
PS D:\-UNIVASF-Compilador-Pascal-TAM> Get-Content exemplos\valido2-tam.txt
PUSH x, y, positivo
LOADL 10
STORE x
LOAD x
LOADL 2
CALL(SB) 10[PB]
LOADL 3
CALL(SB) 7[PB]
STORE y
LOAD y
LOADL 0
CALL(SB) 14[PB]
STORE positivo
LOAD positivo
JUMPIF(0) ELSE_1
LOAD y
LOADL 1
CALL(SB) 7[PB]
STORE y
JUMP FIM_IF_2
ELSE_1:
LOAD y
LOADL 1
CALL(SB) 8[PB]
STORE y
FIM_IF_2:
WHILE_3:
LOAD y
LOADL 0
CALL(SB) 15[PB]
JUMPIF(0) FIM_WHILE_4
LOAD y
LOADL 1
CALL(SB) 8[PB]
STORE y
JUMP WHILE_3
FIM_WHILE_4:
HALT
```

15. CONCLUSÕES FINAIS

15.1. Avaliação do projeto pelo grupo

O *desenvolvimento* do compilador, ao longo das cinco etapas, cumpriu integralmente os objetivos propostos: o produto final lê programas de uma linguagem inspirada em Pascal e os traduz para código da máquina TAM, passando por análise léxica, análise sintática recursiva descendente LL(1), montagem e visualização da AST, análise de contexto com tabela de símbolos e geração de

código com gerador de rótulos, tudo estruturado sobre o padrão de projeto Visitor, conforme exigido. Na avaliação do grupo, o maior ganho de aprendizado veio da obrigação de transformar a teoria em decisões concretas: calcular FIRST e FOLLOW deixou de ser um exercício abstrato quando os conflitos da gramática original impediram, na prática, a escrita do analisador preditivo; a distinção entre sintaxe livre de contexto e dependências de contexto tornou-se palpável ao separar o que o Parser consegue rejeitar do que só o Checker detecta; e o modelo de máquina de pilha se consolidou ao depurar, instrução por instrução, o código gerado para expressões e comandos de controle. A divisão do trabalho por fases, com interfaces combinadas entre os membros (formato do token, contrato do Visitor, estrutura da tabela de símbolos), também exercitou práticas reais de engenharia de software, e a suíte de testes automatizada garantiu que a evolução de uma etapa não quebrasse as anteriores.

15.2. Principais dificuldades encontradas e encaminhamentos correspondentes

A primeira dificuldade relevante foi a adequação da gramática ao método LL(1): a *recursão* à esquerda das expressões e o prefixo comum entre atribuição e chamada de procedimento exigiram estudo das transformações (eliminação de recursão, fatoração) e a decisão documentada sobre o dangling else. O encaminhamento foi reescrever a gramática em EBNF iterativa e validar cada ponto de decisão com os conjuntos FIRST/FOLLOW antes de codificar. A segunda dificuldade foi definir os casos omissos da especificação, decidiu-se, e documentou-se, que subprogramas não teriam parâmetros, que o escopo seria único e global e que literais de texto seriam rejeitados na análise de contexto; registrar essas decisões por escrito evitou retrabalho nas etapas seguintes. A terceira foi a integração entre as fases: acordar cedo o contrato do token (tipo, lexema, linha, coluna) e a interface Visitor permitiu que léxico, sintático, contexto e geração evoluíssem em paralelo. Por fim, na geração de código, a escolha de rótulos simbólicos (em vez de endereços numéricos com backpatching) simplificou a implementação e tornou o código-objeto legível, ao custo de exigir que um eventual montador/interpretador resolva os rótulos.

15.3. Críticas e sugestões

Como críticas construtivas ao próprio produto, o grupo registra: a ausência de uma opção de linha de comando que execute apenas a análise de contexto (hoje embutida em `--code`), a recuperação de erros limitada ao modo pânico (um erro por execução) e a ausência de comandos de entrada e saída (`read/write`), que tornariam os programas de teste mais expressivos. Como sugestões de evolução: implementar parâmetros e variáveis locais em subprogramas, com pilha de escopos e quadros de ativação na TAM; adicionar um interpretador da máquina TAM ao projeto, permitindo executar o código gerado e fechar o ciclo compilação–execução; acrescentar a opção `--check` (contexto sem geração); e evoluir as mensagens de erro para sugerir correções (por exemplo, identificadores semelhantes ao não declarado). Em relação à disciplina, o grupo sugere manter o formato incremental em etapas com entregas parciais, que se mostrou eficaz para distribuir o esforço, e considerar a disponibilização de um interpretador TAM de referência para validação automática do código gerado.